

Advanced architectures: regression, composites, hypervectors, multioutput, auto-encoding, and graphs

Ole-Christoffer Granmo

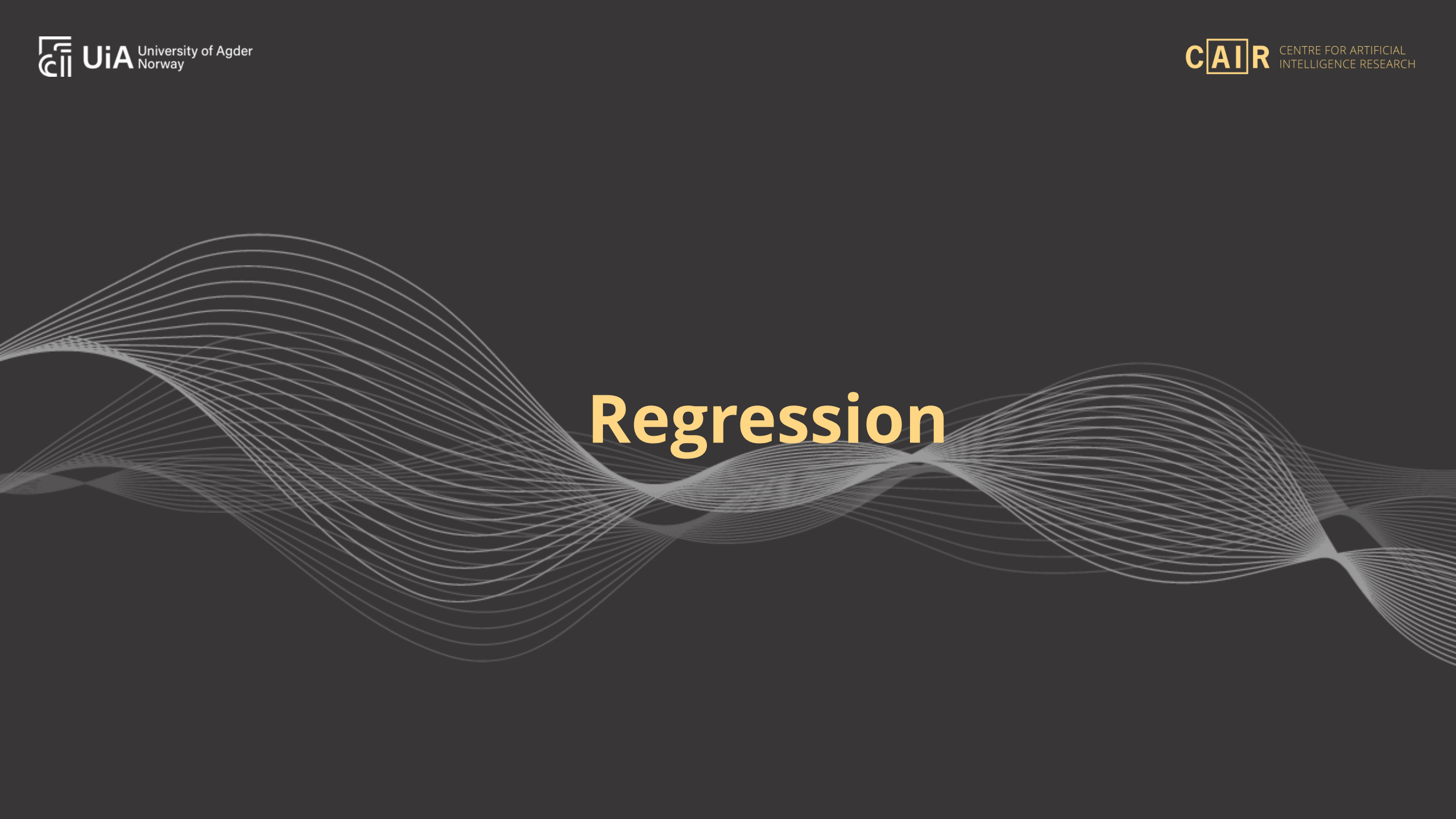
Centre for AI Research

University of Agder

Norway

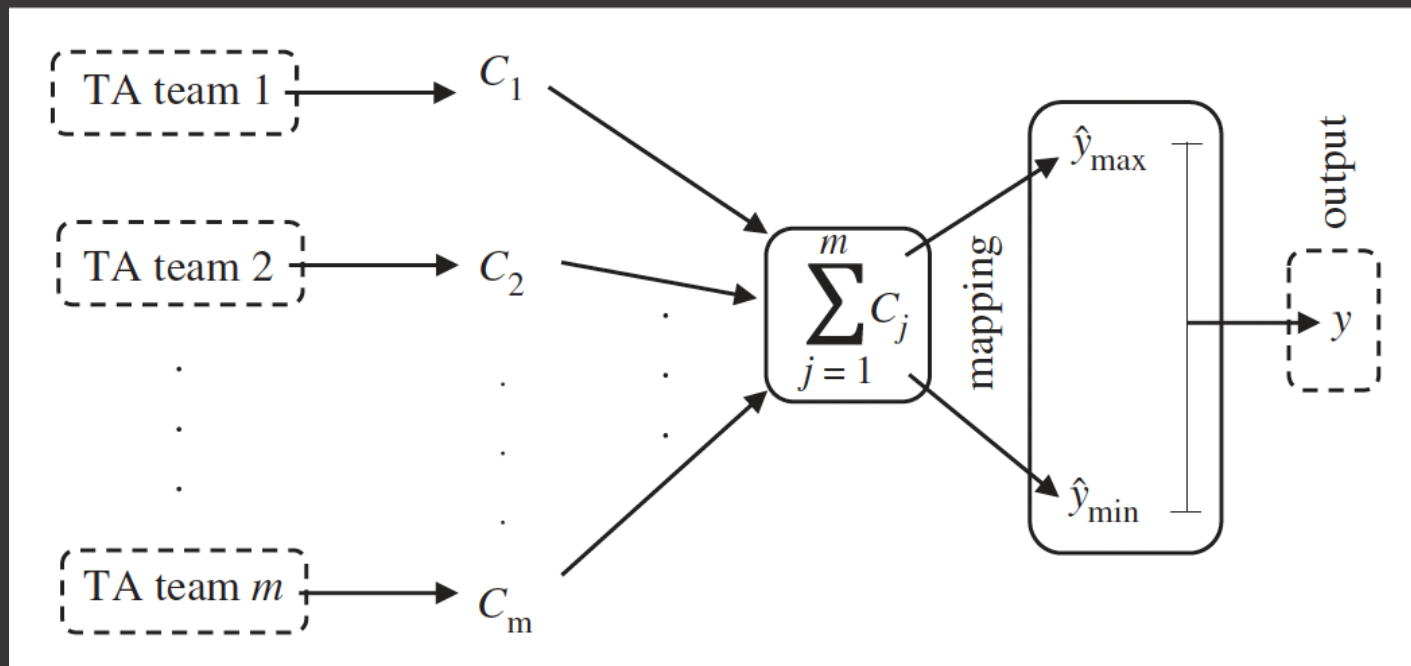
Overview

- Regression
- Composites
- Hypervectors
- Multioutput with clause sharing
- Auto-encoding and logical word-embedding
- Graph Tsetlin machine



Regression

Regression Tsetlin machine



K. Darshana Abeyrathna, Ole-Christoffer Granmo, Xuan Zhang, Lei Jiao, and Morten Goodwin. The Regression Tsetlin Machine - A Novel Approach to Interpretable Non-Linear Regression. Philosophical Transactions of the Royal Society A, 378, 2019.

Regression Tsetlin machine

101	$(1 * *)$	$4 \times (1 * *) \times 1 = 4$	$5 \times 100 = 500$
	$(* * 1)$	$2 \times (* 1 *) \times 0 = 0$	
		$1 \times (* * 1) \times 1 = 1$	
110	$(1 * *)$	$4 \times (1 * *) \times 1 = 4$	$6 \times 100 = 600$
	$(* 1 *)$	$2 \times (* 1 *) \times 1 = 2$	
		$1 \times (* * 1) \times 0 = 0$	
111	$(1 * *)$	$4 \times (1 * *) \times 1 = 4$	$7 \times 100 = 700$
	$(* 1 *)$	$2 \times (* 1 *) \times 1 = 2$	
	$(* * 1)$	$1 \times (* * 1) \times 1 = 1$	

Regression Tsetlin machine

model	dengue incidences	energy performance	stock selection		real estate valuation	aerofoil noise
			annual return	real win rate		
RTM	5.184	0.503 ± 0.00	0.077 ± 0.00	0.089 ± 0.00	5.218 ± 0.00	2.244 ± 0.00
RT	7.769	0.598 ± 0.00	0.091 ± 0.00	0.096 ± 0.00	6.033 ± 0.11	2.280 ± 0.07
RF	9.122	0.563 ± 0.00	0.088 ± 0.00	0.097 ± 0.00	5.360 ± 0.00	1.921 ± 0.00
SVR	5.305	1.155 ± 0.00	0.073 ± 0.00	0.092 ± 0.00	5.697 ± 0.00	2.156 ± 0.00

```
from tmu.models.regression.vanilla_regressor import TMRegressor
from pyTsetlinMachine.tools import Binarizer
import numpy as np
from time import time

from sklearn import datasets
from sklearn.model_selection import train_test_split

california_housing = datasets.fetch_california_housing()
X = california_housing.data
Y = california_housing.target

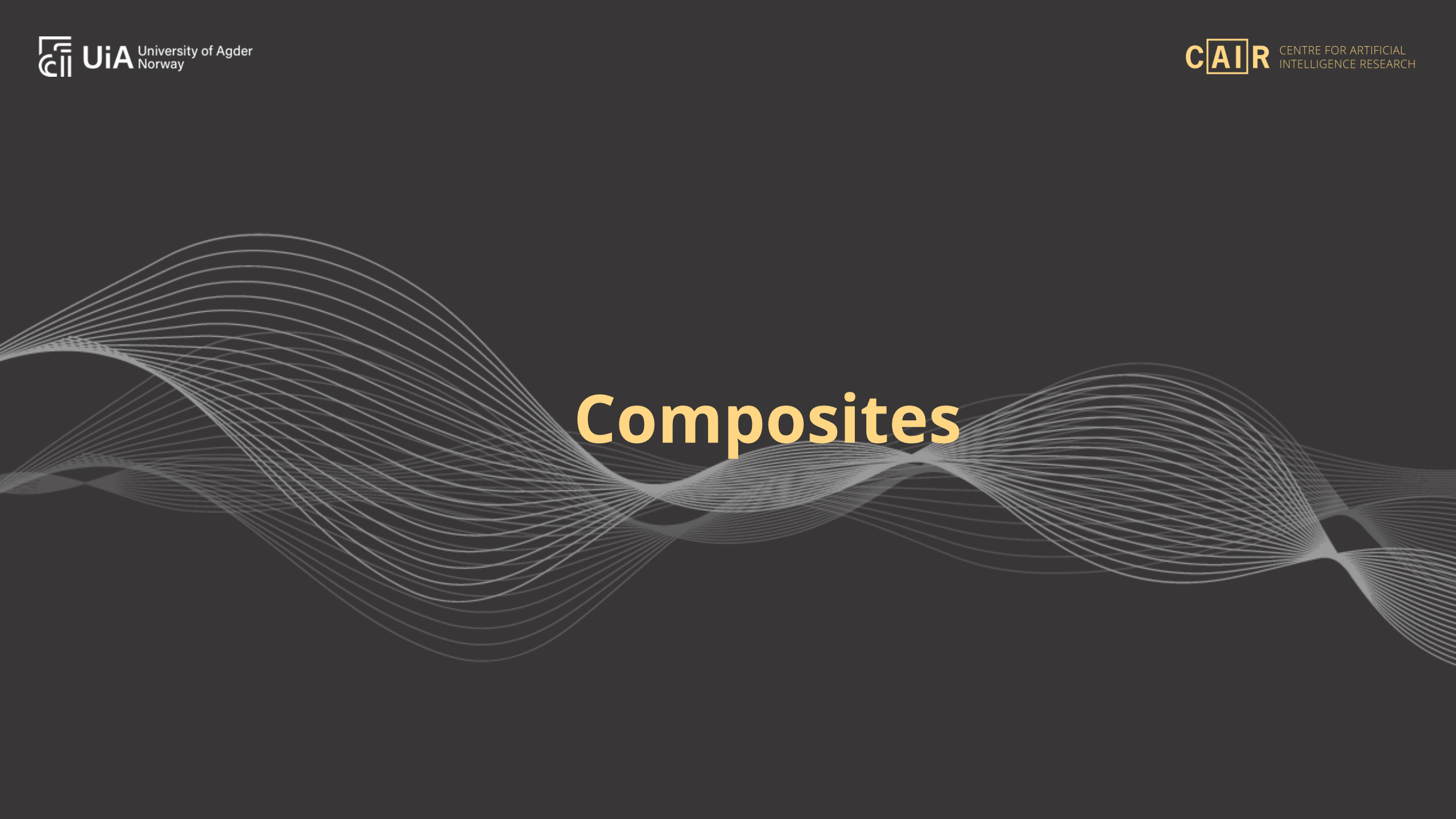
b = Binarizer(max_bits_per_feature = 10)
b.fit(X)
X_transformed = b.transform(X)

print("\nRMSD over 25 runs:\n")
tm_results = np.empty(0)
for i in range(25):
    X_train, X_test, Y_train, Y_test = train_test_split(X_transformed, Y)

    tm = TMRegressor(1000, 500*10, 2.75, platform='CPU', weighted_clauses=True)

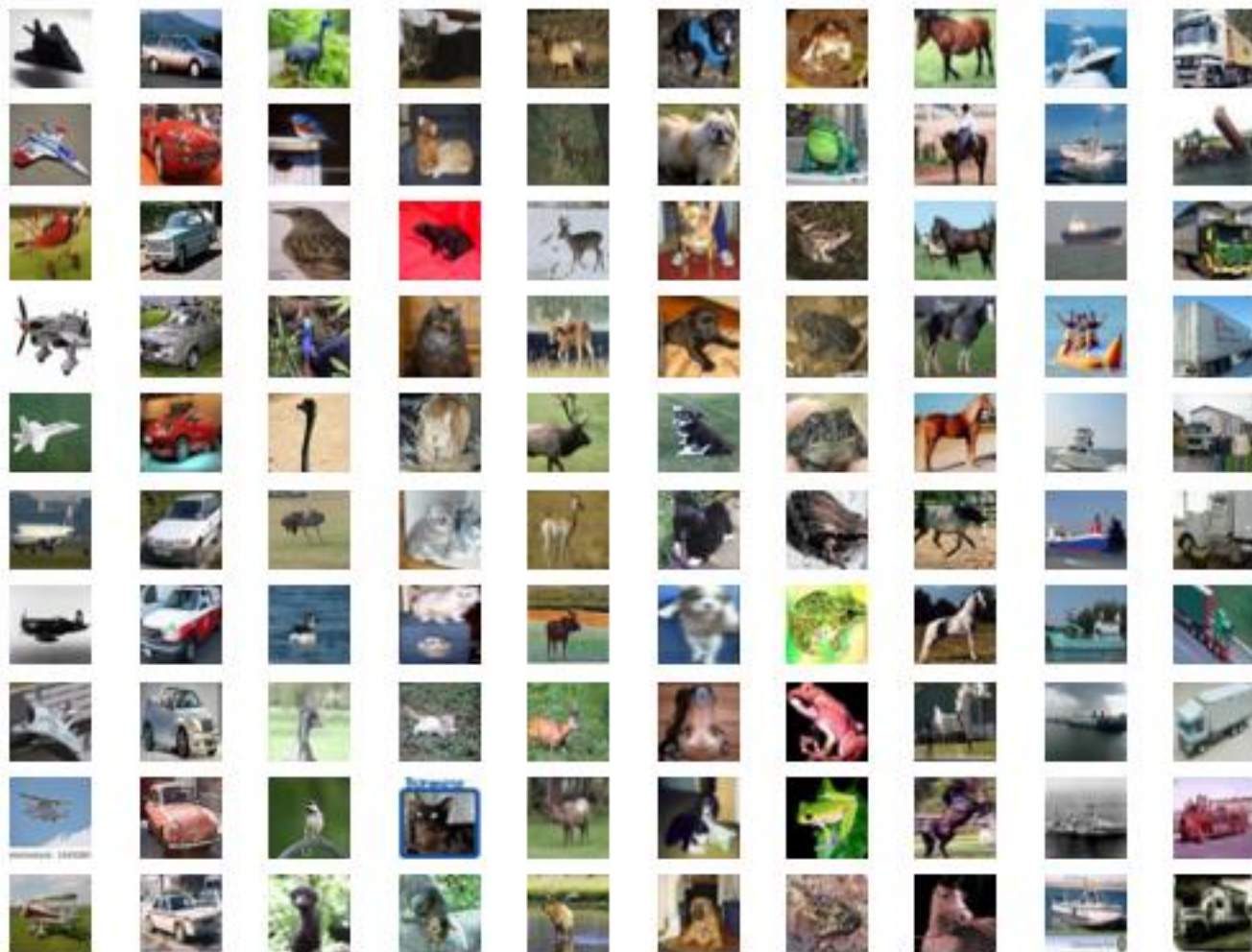
    start = time()
    for e in range(30):
        tm.fit(X_train, Y_train)
    stop = time()
    tm_results = np.append(tm_results, np.sqrt(((tm.predict(X_test) - Y_test)**2).mean()))

print("#%d RMSD: %.2f +/- %.2f (%.2fs)" % (i+1, tm_results.mean(), 1.96*tm_results.std()/np.sqrt(i+1), stop-start))
```

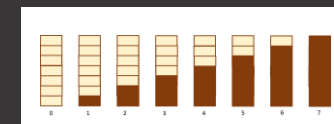


Composites

plane car bird cat deer dog frog horse ship truck

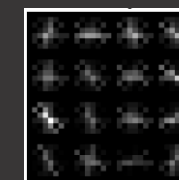


10x10 Adaptive Thresholding: 57.0%



4x4 Color Thermometers: 62.7%

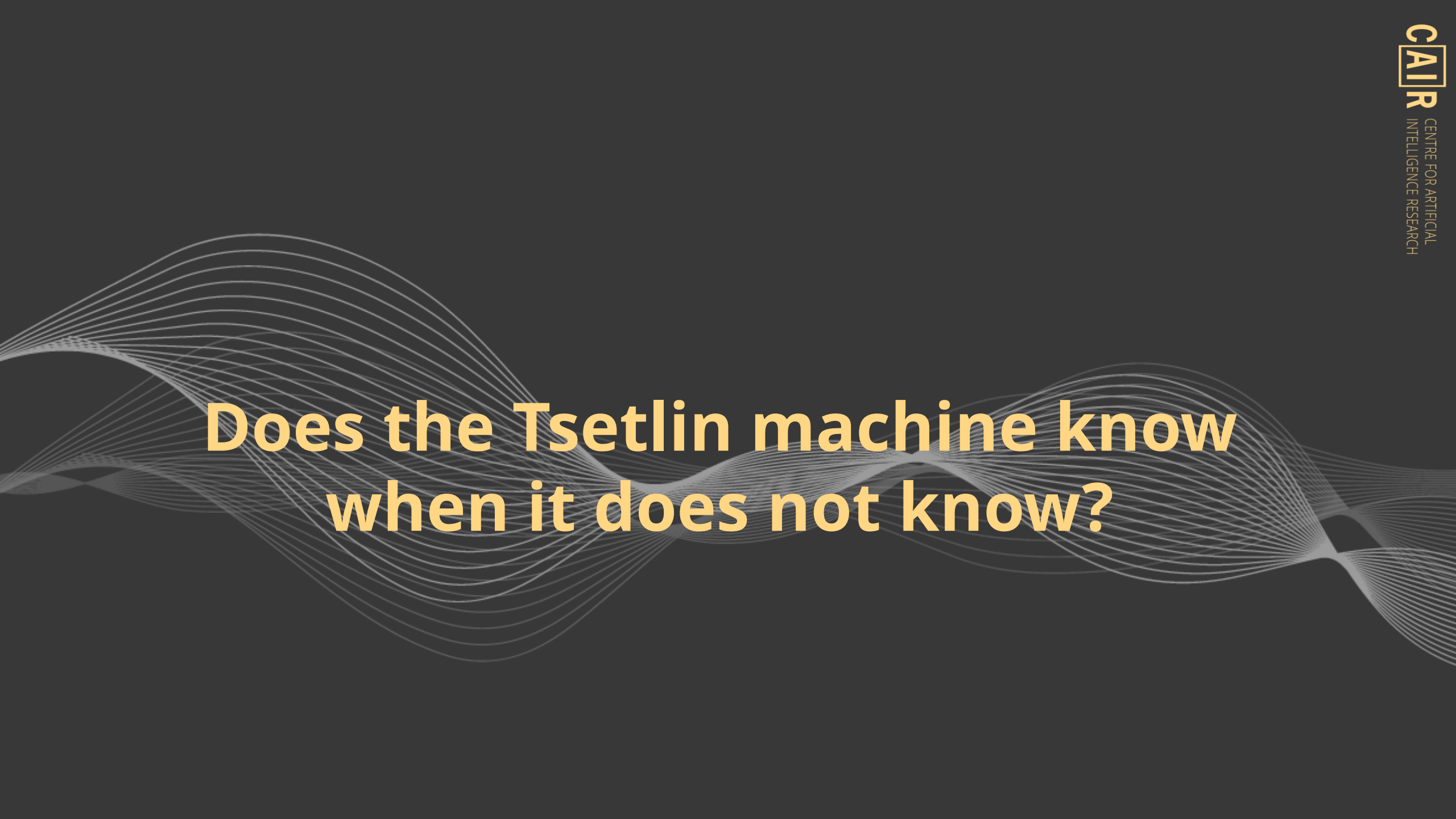
3x3 Color Thermometers: 62.1%



Histogram of Gradients: 63.5%

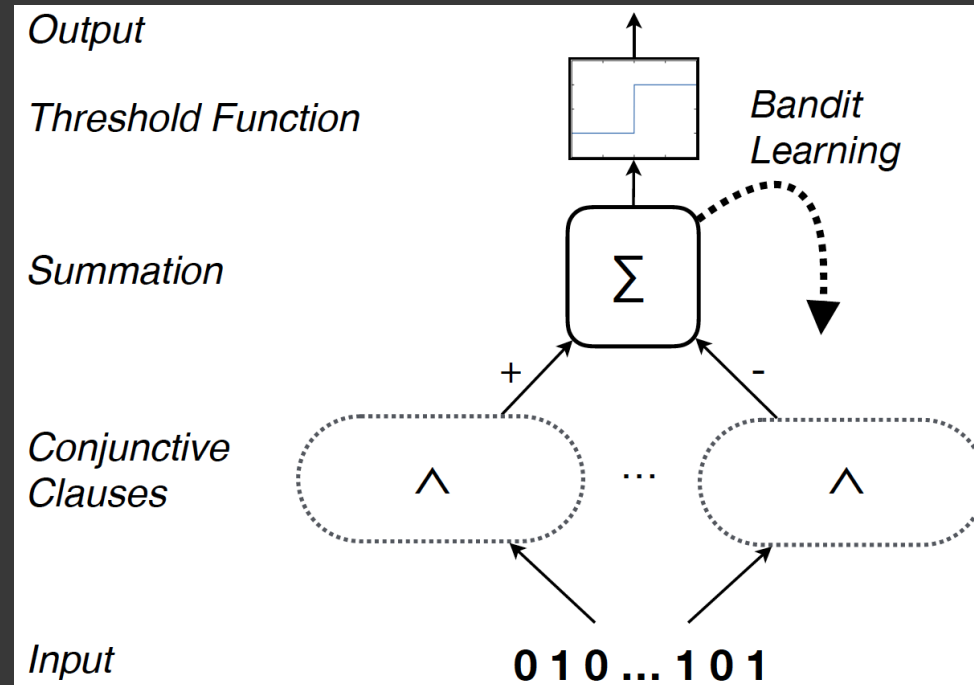






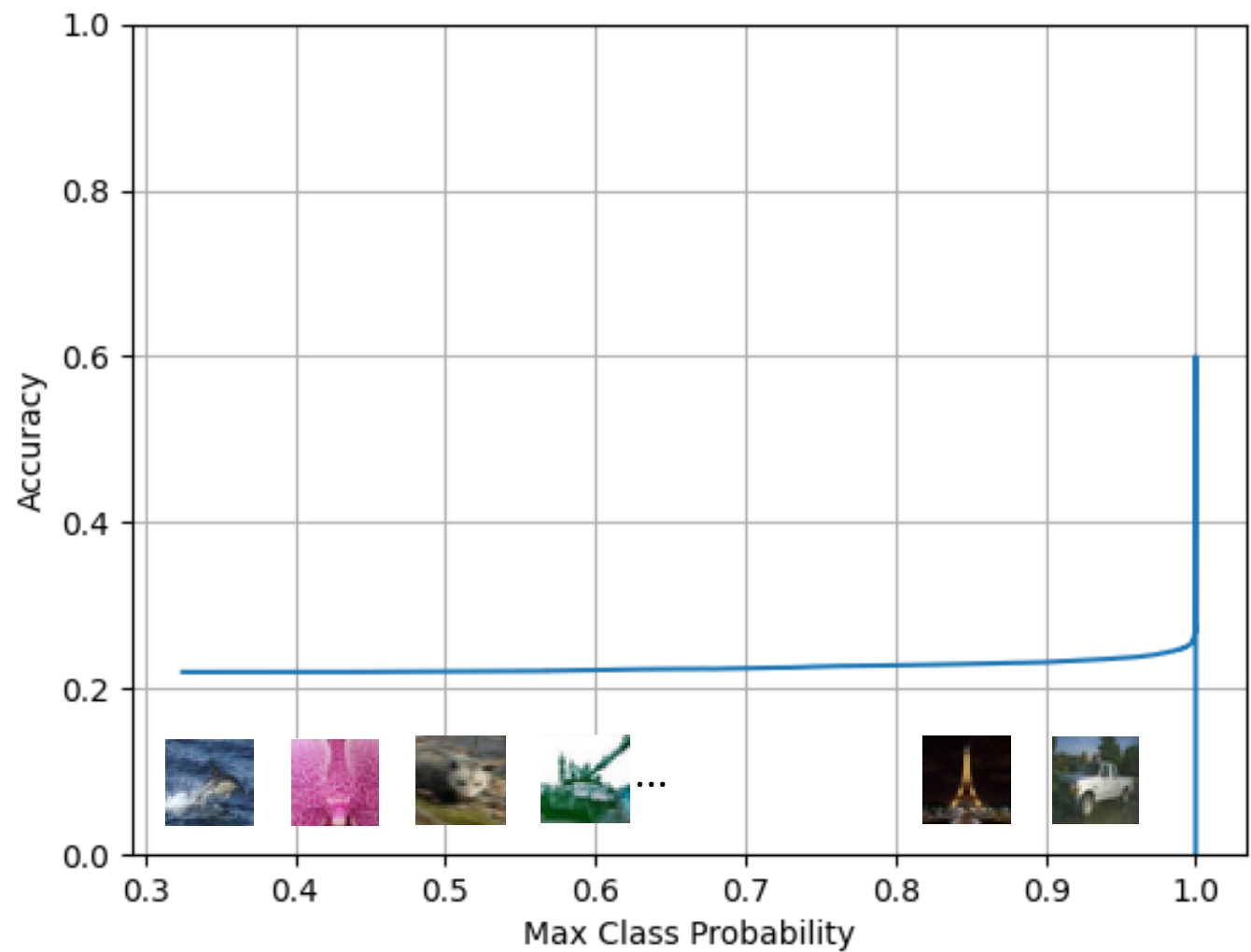
**Does the Tsetlin machine know
when it does not know?**

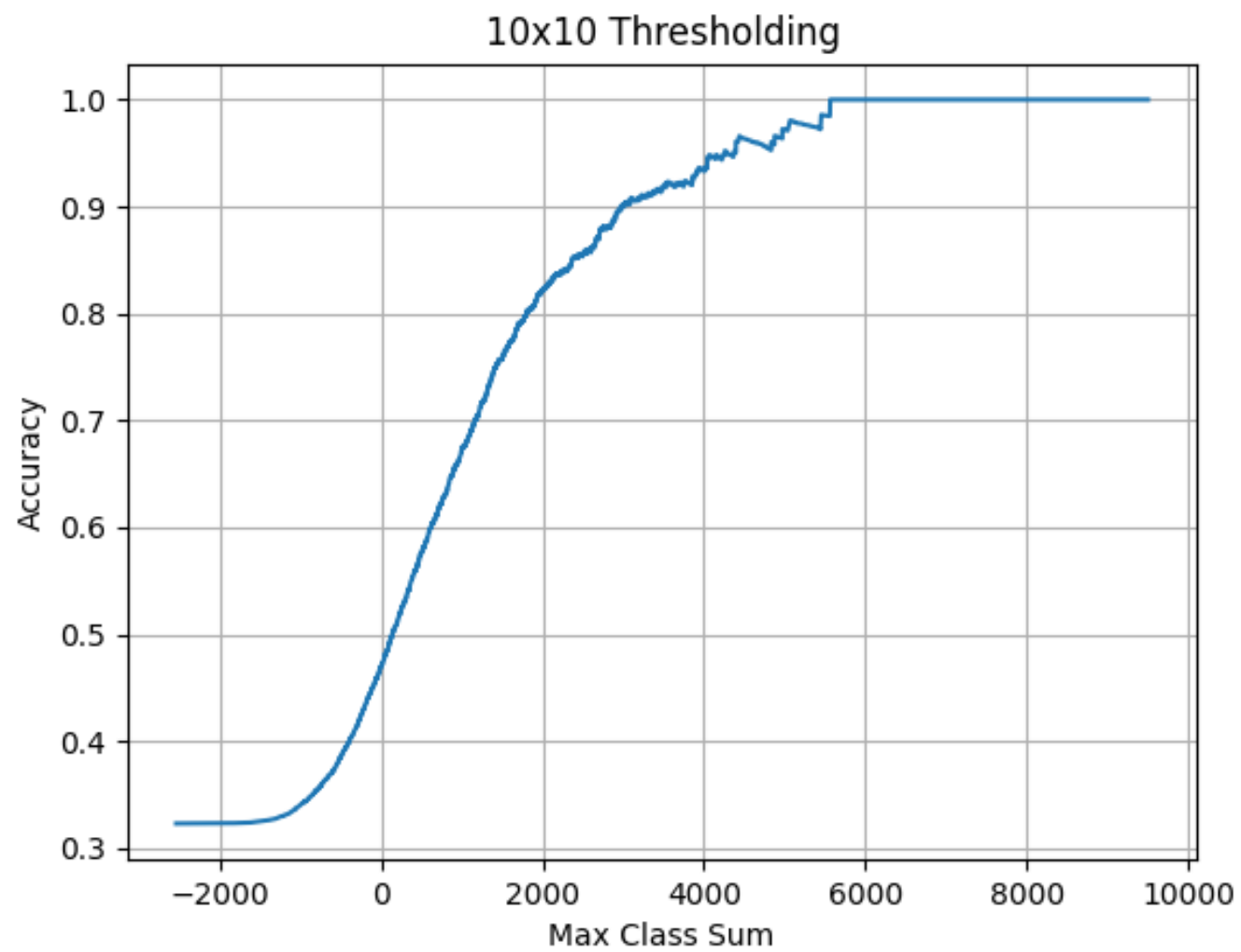
Tsetlin machine classification confidence

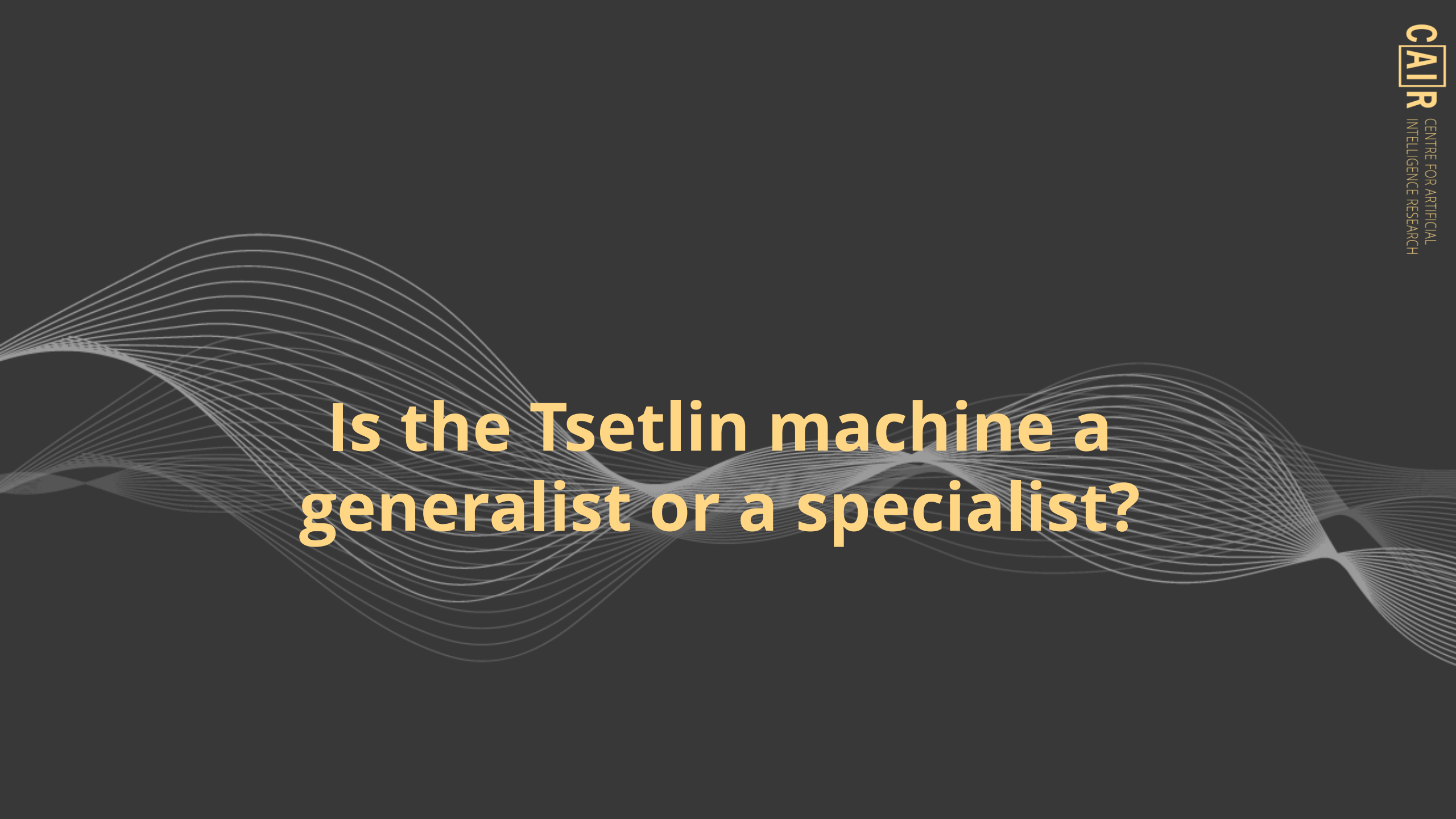


K. D. Abeyrathna, O. -C. Granmo and M. Goodwin, "On Obtaining Classification Confidence, Ranked Predictions and AUC with Tsetlin Machines," 2020 IEEE Symposium Series on Computational Intelligence (SSCI)

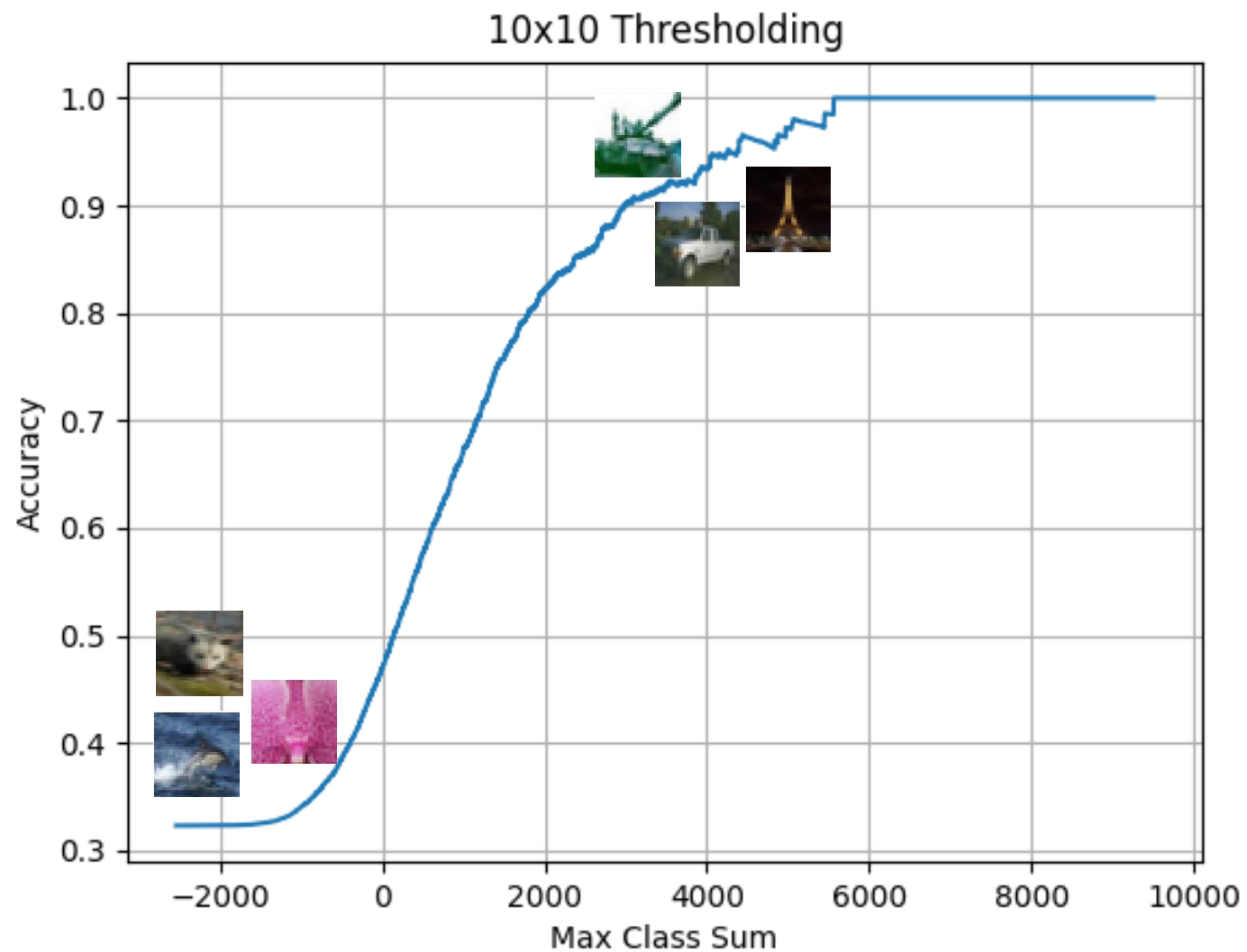
Class sum:
$$\left(\sum_{j=1}^{n/2} C_j^+(X) \right) - \left(\sum_{j=1}^{n/2} C_j^-(X) \right)$$

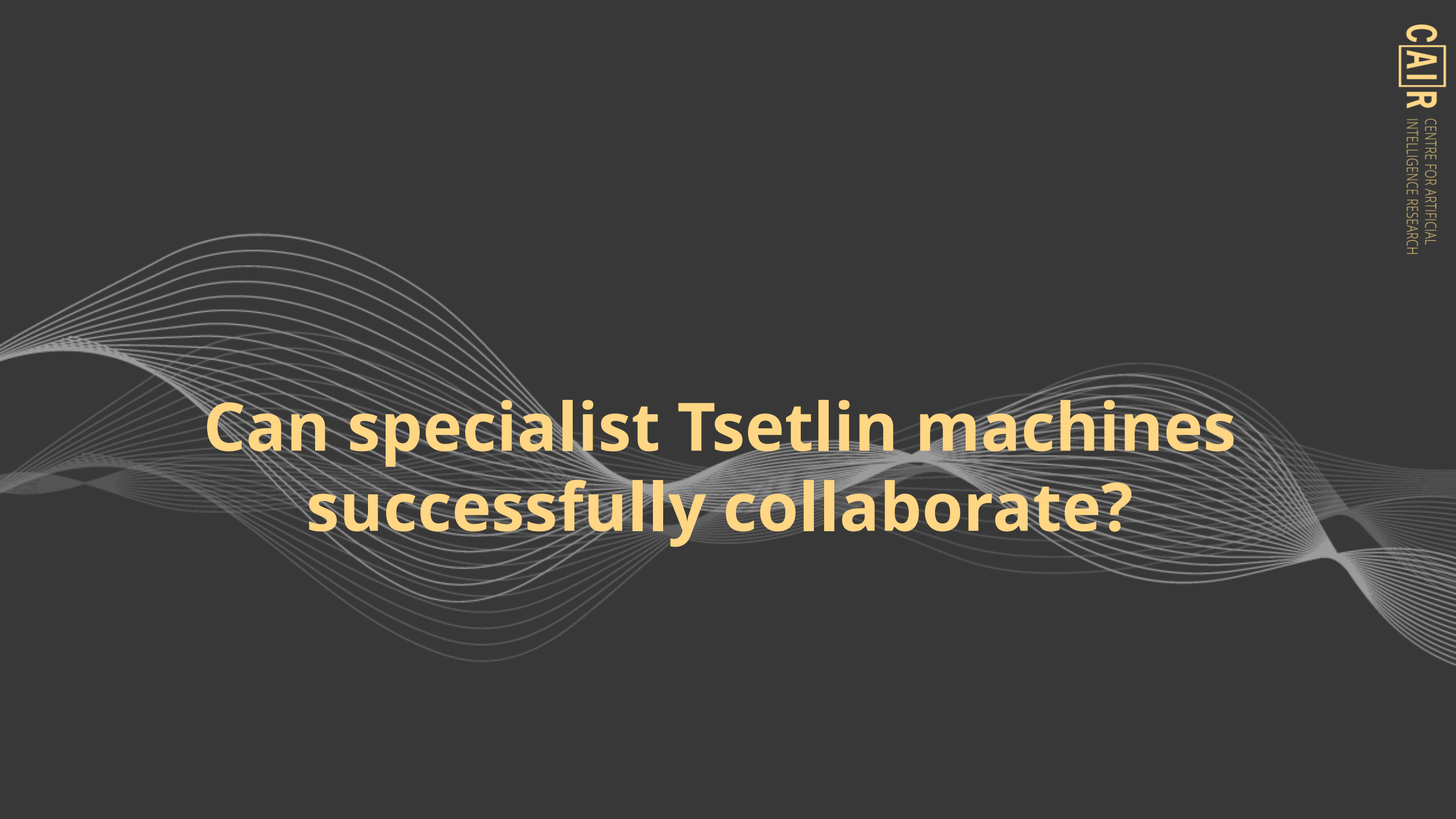




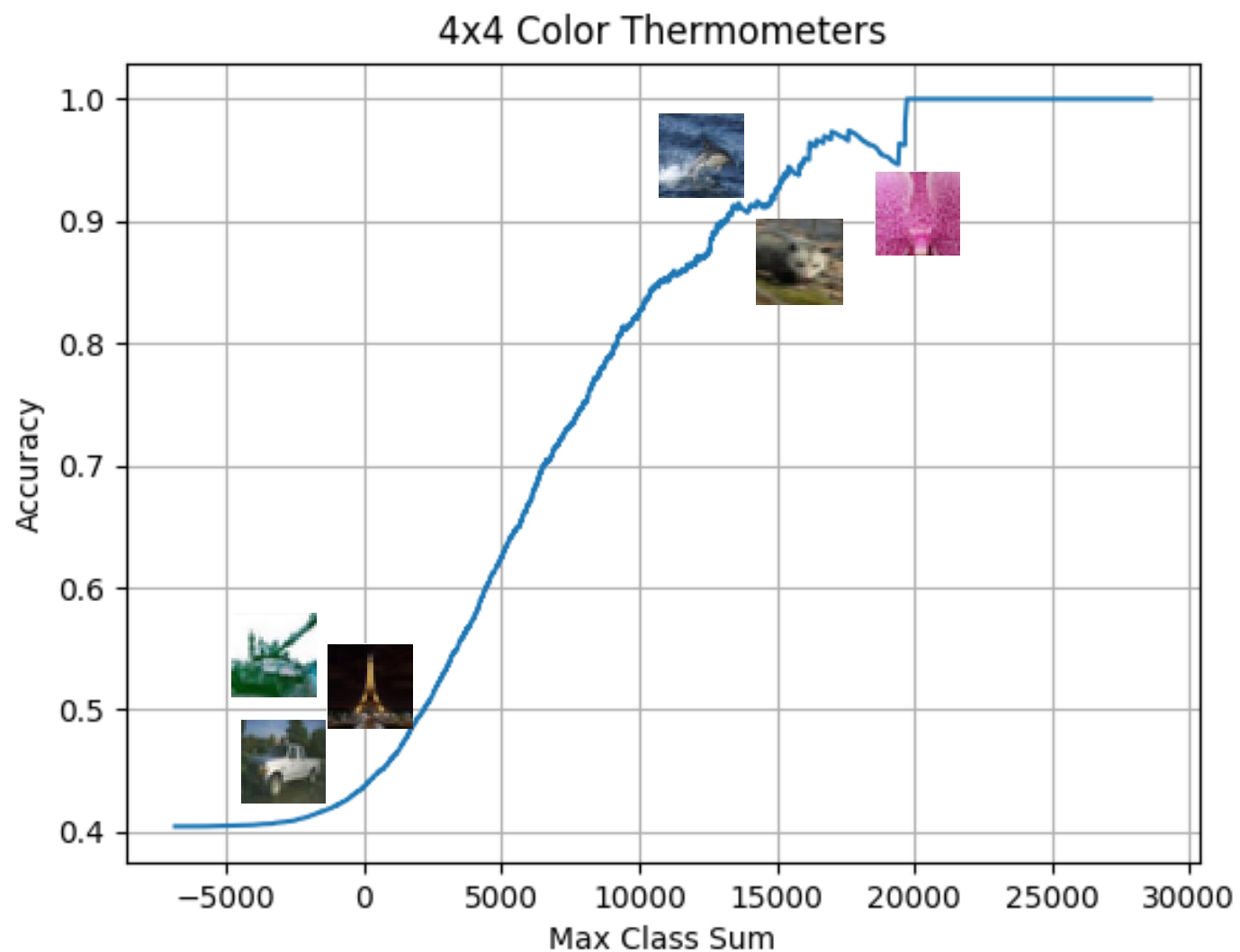


**Is the Tsetlin machine a
generalist or a specialist?**

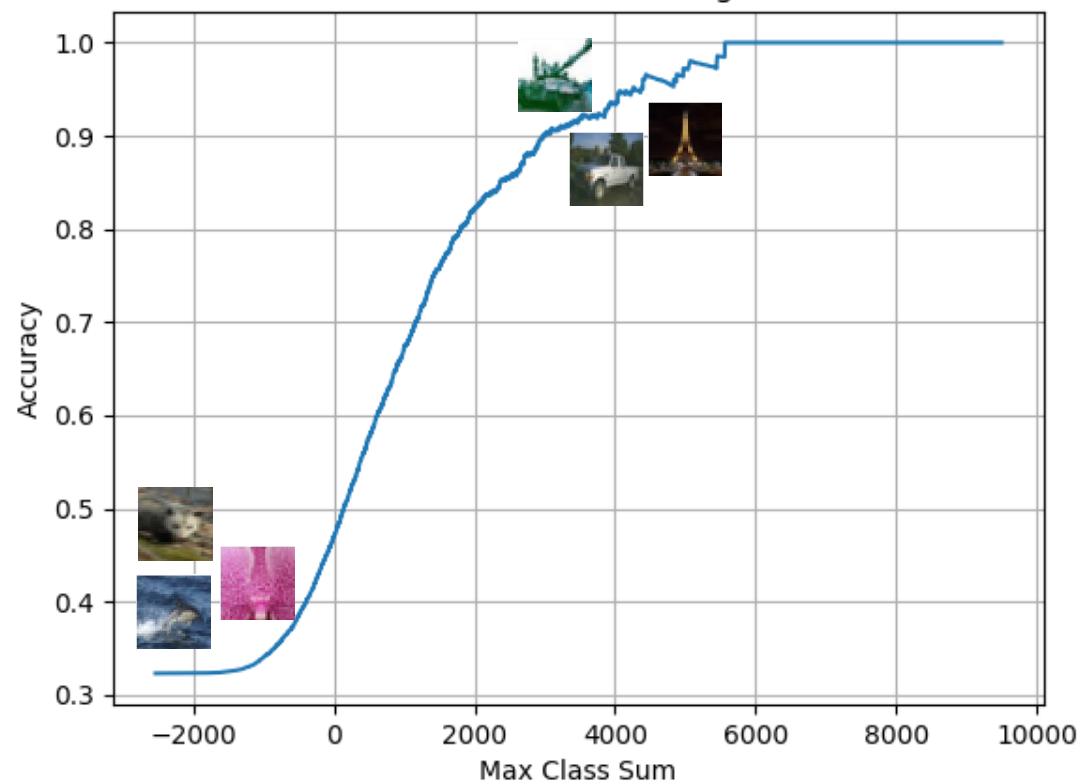




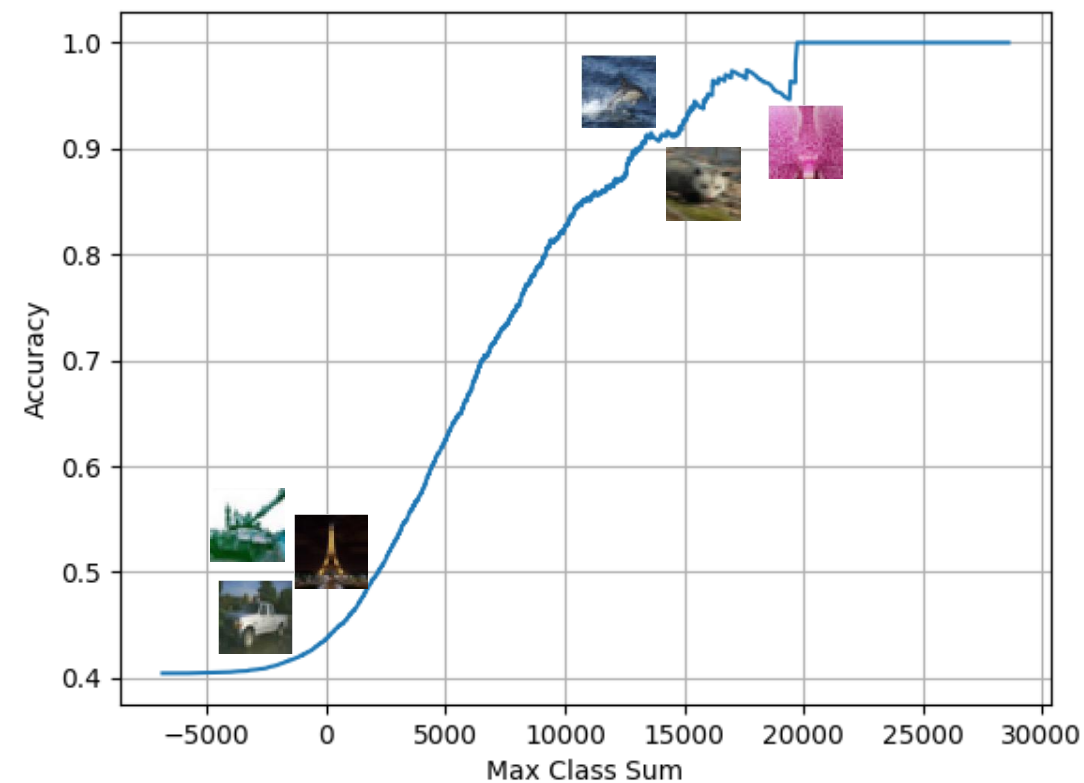
**Can specialist Tsetlin machines
successfully collaborate?**



10x10 Thresholding



4x4 Color Thermometers



Confidence

- Did the student follow the lectures of the course?
- Did the student complete the textbook?
- Did the student solve the assignments?

R1 if *Lectures* **then** *Pass*,

R2 if *Textbook* **then** *Pass*,

R3 if **not** *Assignments* **then** *Fail*.

Confidence

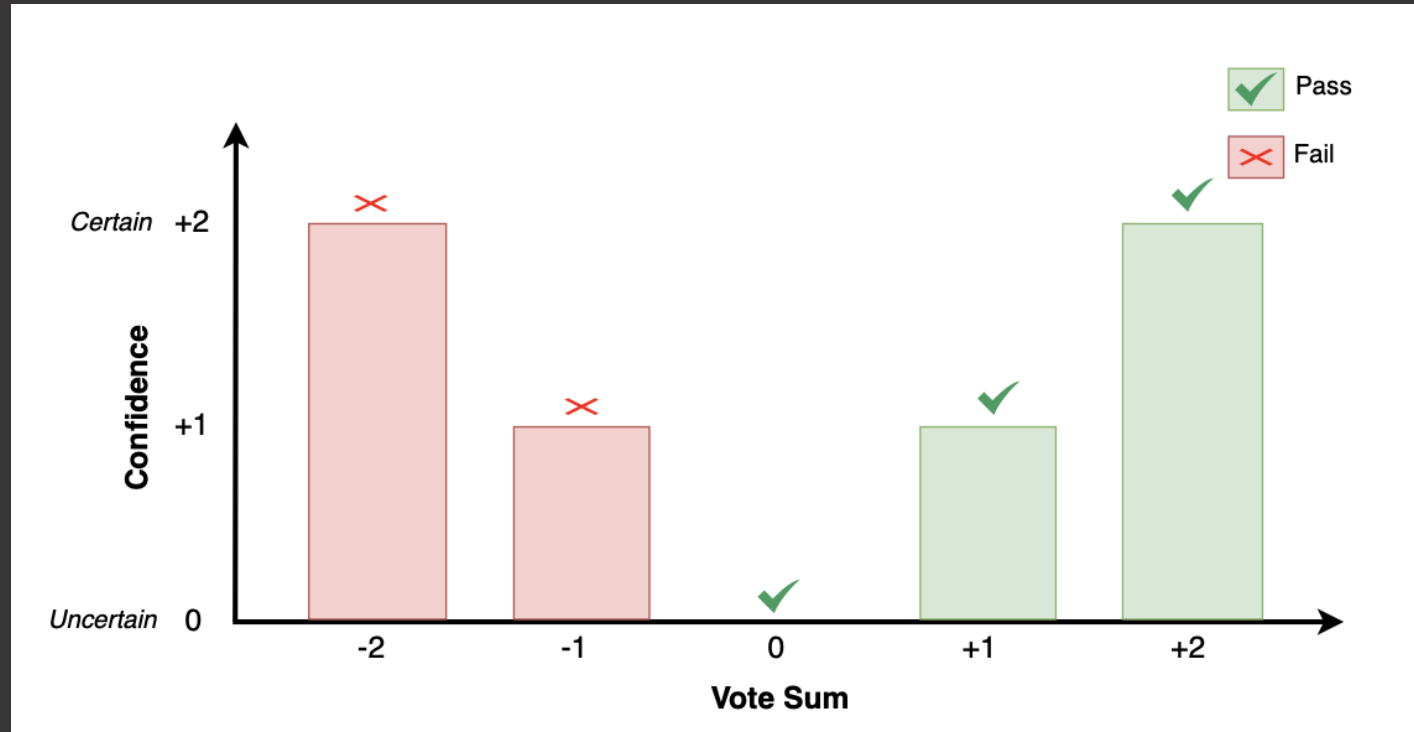
#	Lectures	Textbook	Assignments	<i>Pass</i>	R1	R2	R3	v
1.	yes	yes	yes	<i>yes</i>	●	●	·	+2
2.	no	yes	no	<i>no</i>	·	●	●	+0
3.	yes	yes	yes	<i>yes</i>	●	●	·	+2
4.	no	no	no	<i>no</i>	·	·	●	-1
5.	yes	yes	no	<i>yes</i>	●	●	●	+1
6.	no	no	no	<i>no</i>	·	·	●	-1

R1 if *Lectures* **then** *Pass*,

R2 if *Textbook* **then** *Pass*,

R3 if **not** *Assignments* **then** *Fail*.

Confidence



Confidence Calculation. In general, it is the absolute value of the vote sum that measures how confident a Tsetlin machine is:

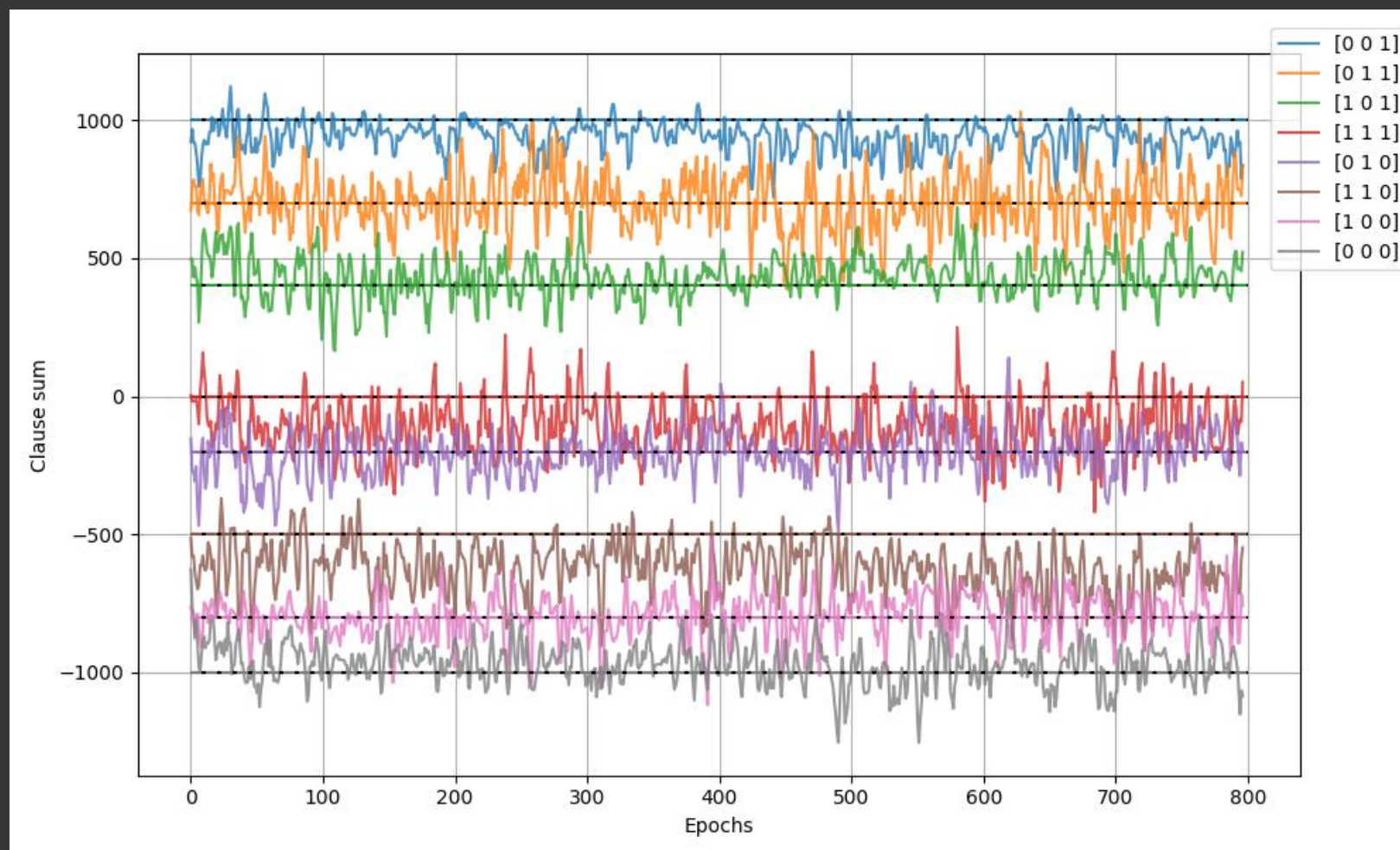
$$\text{Confidence} := \text{abs}(\text{Vote Sum}). \quad (7.1)$$

Trustworthiness

Performance		
Confidence	Low	High
Low	<i>Justifiably Uncertain</i>	<i>Underconfident</i>
High	<i>Overconfident</i>	<i>Justifiably Confident</i>

Measuring trustworthiness

A	B	C	P(X)
0	0	1	1.00
0	1	1	0.85
1	0	1	0.70
1	1	1	0.50
0	1	0	0.40
1	1	0	0.25
1	0	0	0.10
0	0	0	0.00



How can multiple Tsetlin machines collaborate?

Different Populations. Two hospitals may have trained their own Tsetlin machines to predict breast cancer recurrence. Differences in patient demographics, treatment approaches, and facilities will give the two Tsetlin machines unique perspectives.

Different Modalities. Tsetlin machines can also have different views on the patient. One Tsetlin machine may operate on blood samples, another on mammograms, and a third on the narrative written by the doctor. Each source of information offers valuable information on breast cancer recurrence.

Two complementary Tsetlin machines

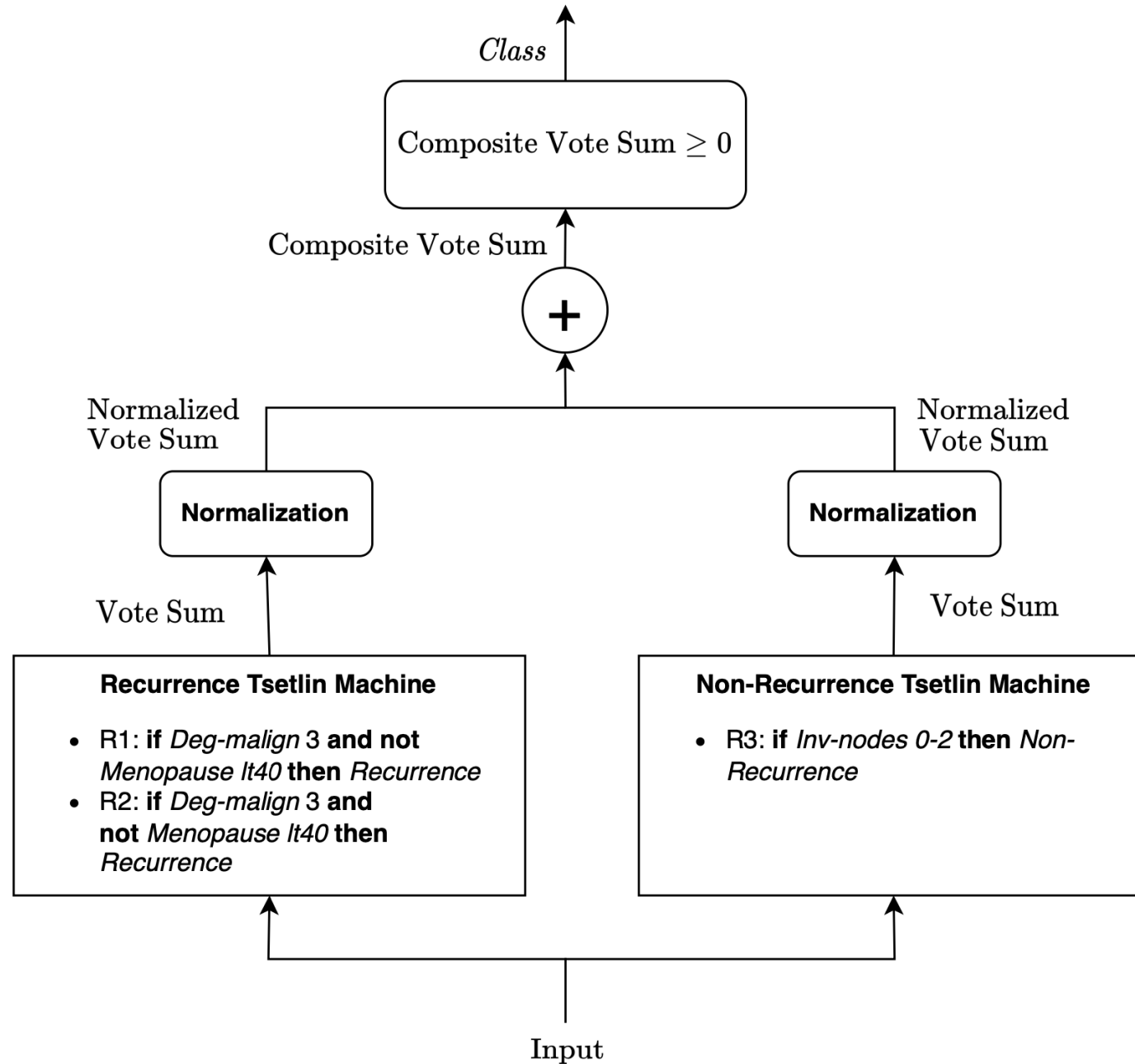
- **Recurrence Tsetlin Machine.** This Tsetlin machine has only trained on recurrence patients, obtaining the two rules:

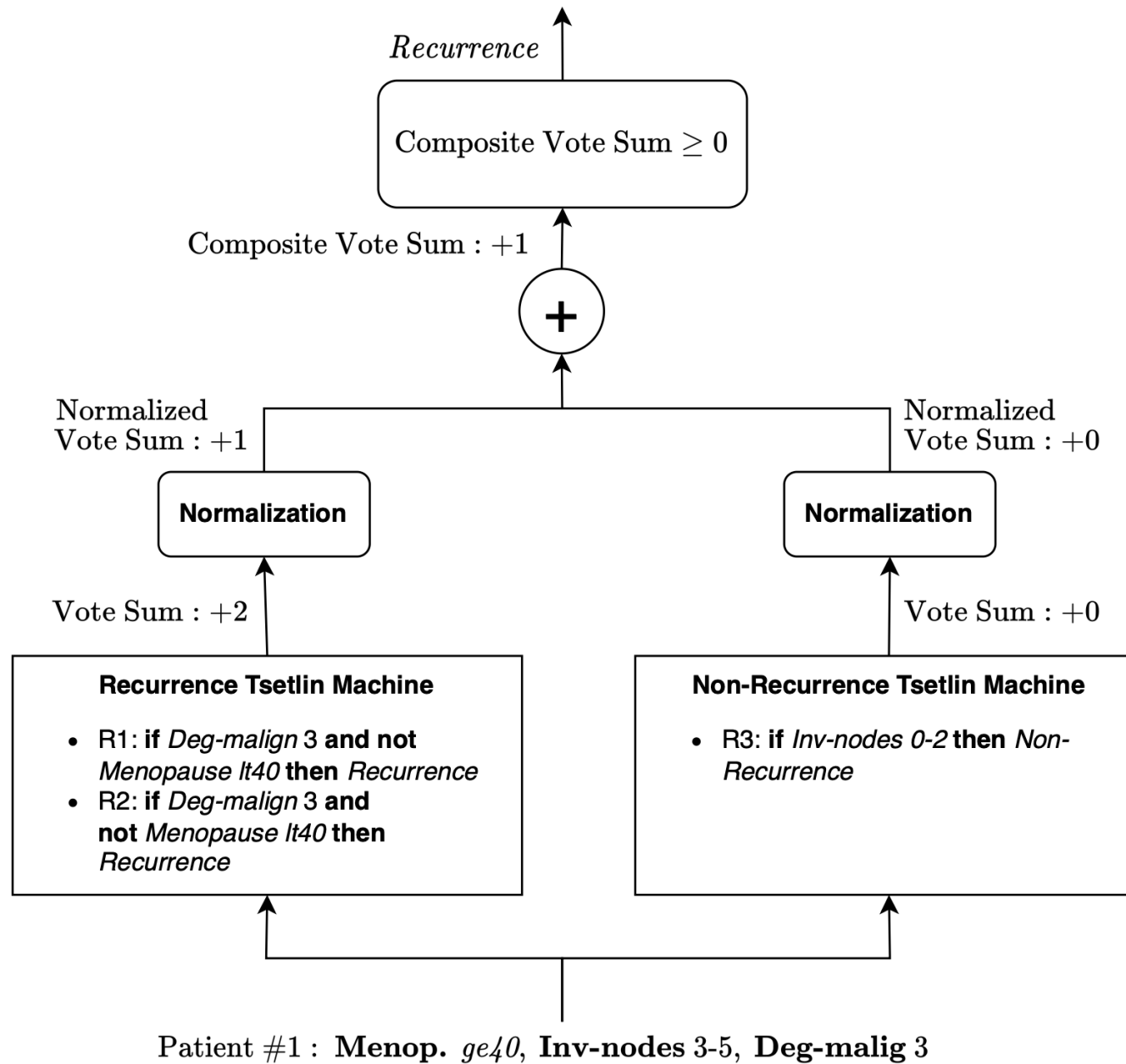
R1 if *Deg-malign 3* and not *Menopause lt40* then *Recurrence*,

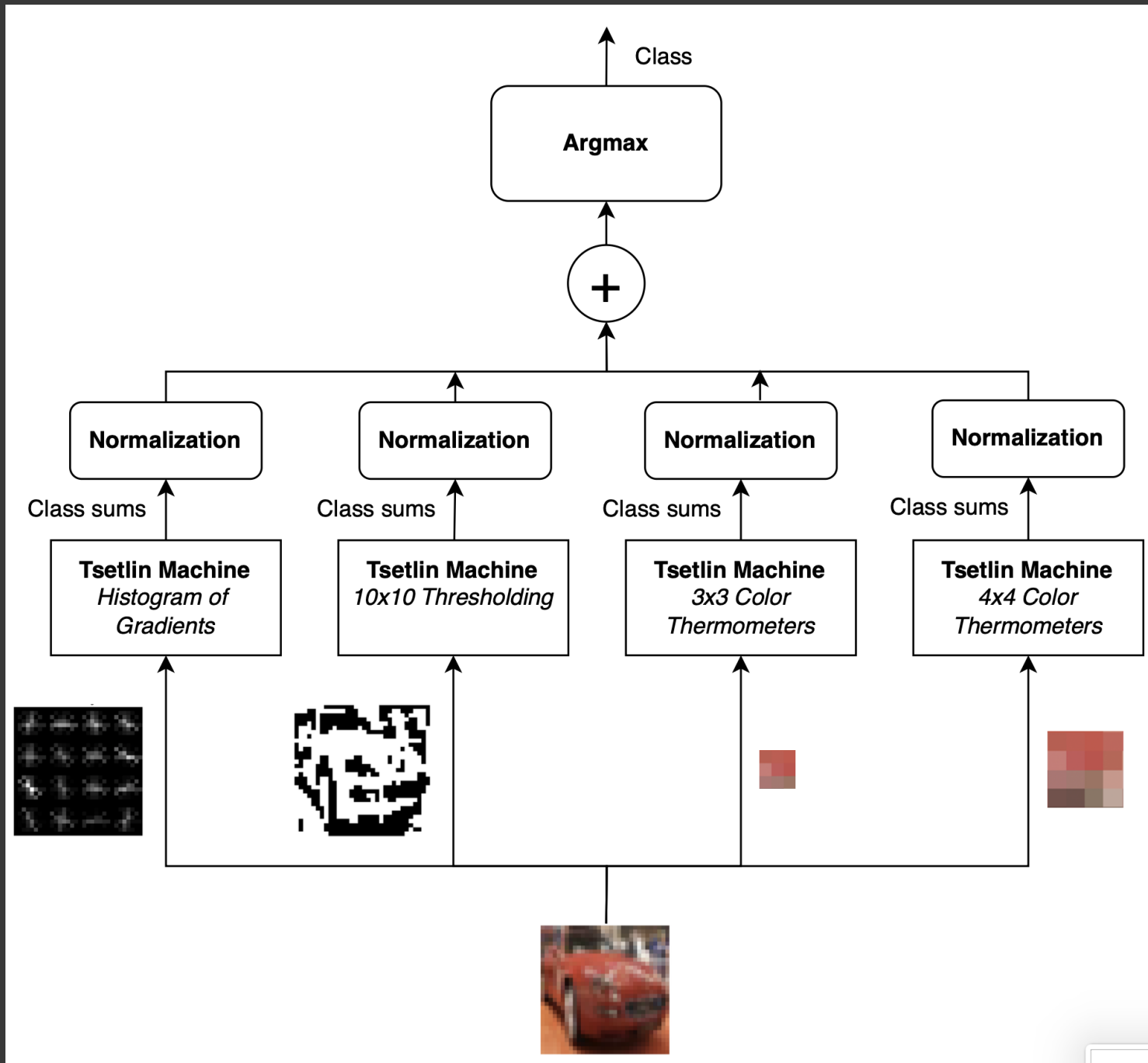
R2 if *Deg-malign 3* and not *Menopause lt40* then *Recurrence*.

- **Non-Recurrence Tsetlin Machine.** This Tsetlin machine can only recognize non-recurrence patients using the following rule:

R3 if *Inv-nodes 0-2* then *Non-Recurrence*.

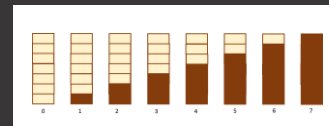
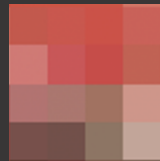






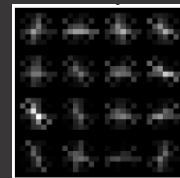


10x10 Adaptive Thresholding: 57.0%



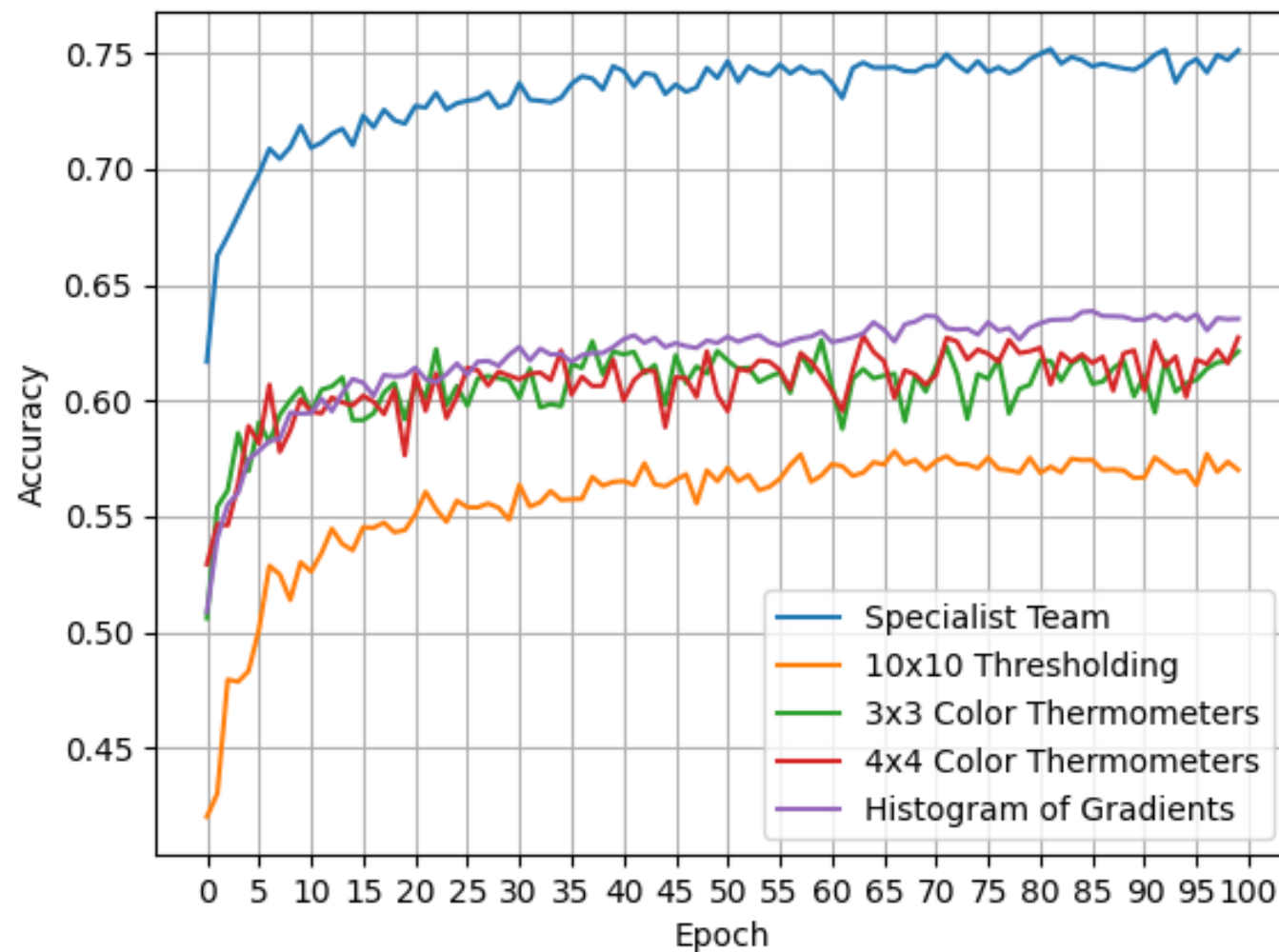
4x4 Color Thermometers: 62.7%

3x3 Color Thermometers: 62.1%



Histogram of Gradients: 63.5%

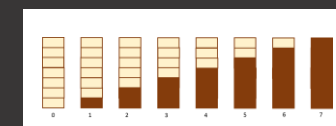
75.1%



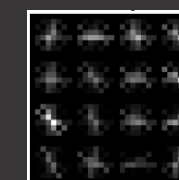
CIFAR-100



10x10 Adaptive Thresholding: 32.3%

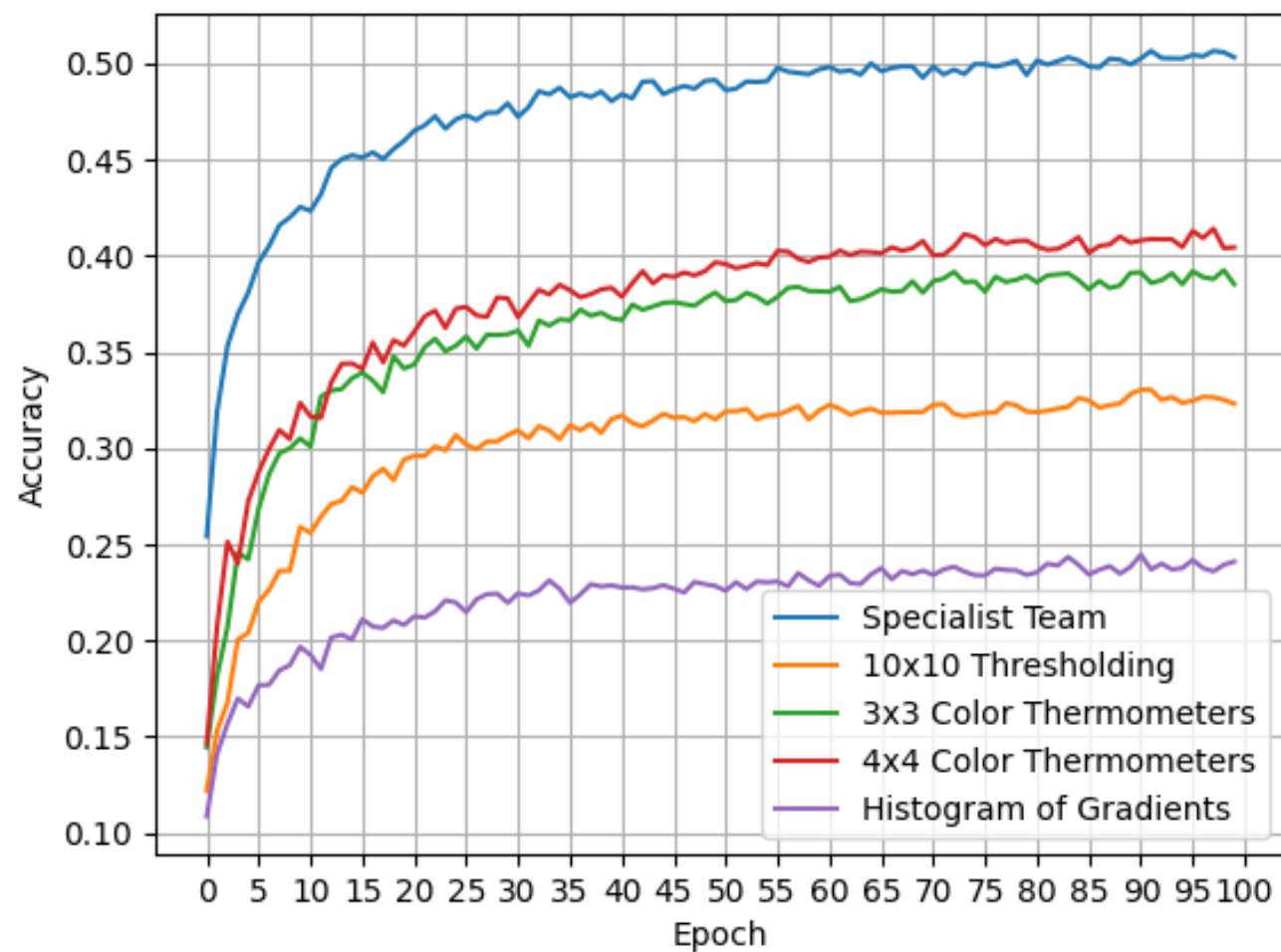


4x4 Color Thermometers: 40.5%
3x3 Color Thermometers: 38.5%



Histogram of Gradients: 24.1%

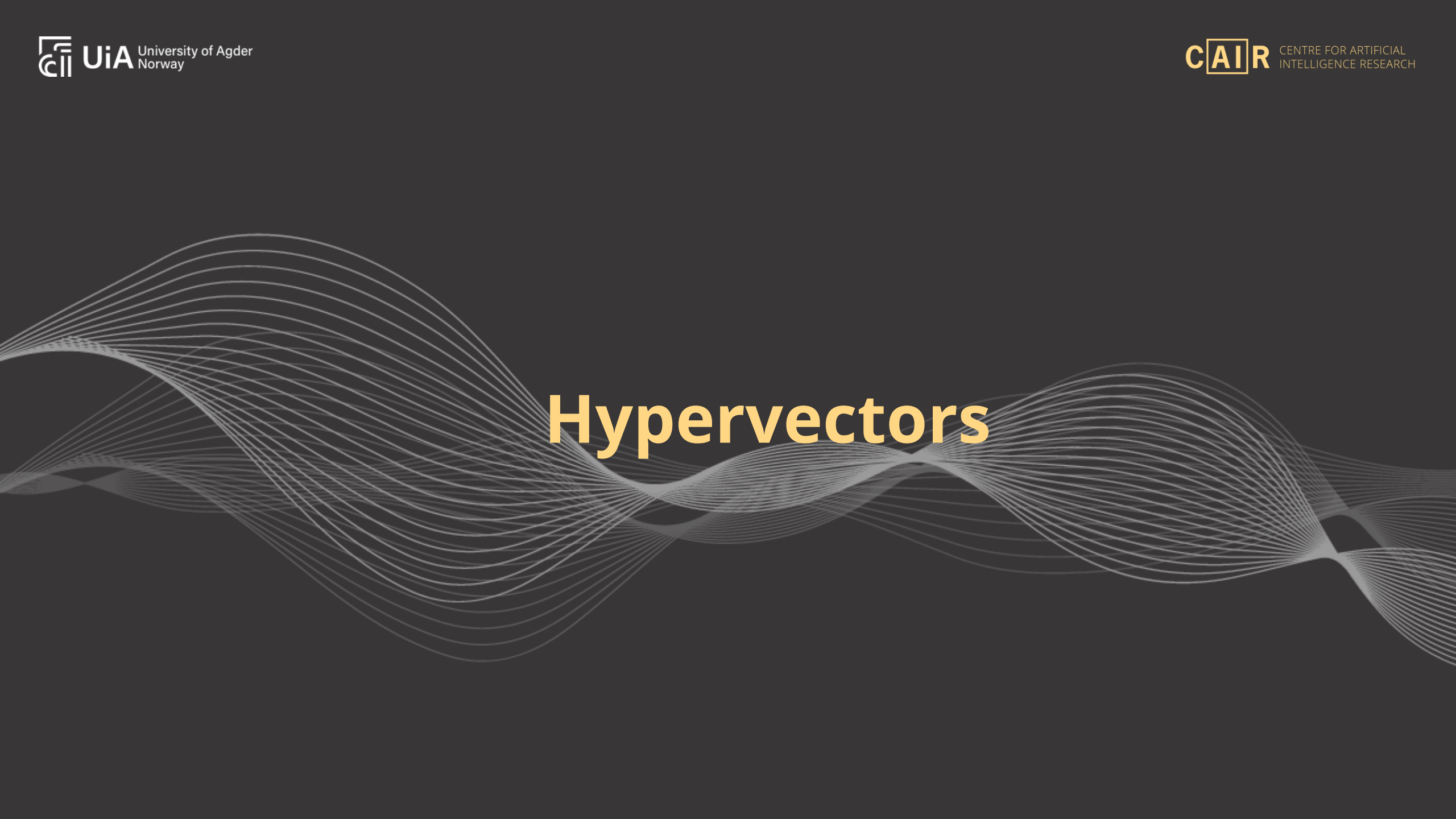
52.2%



TM Specialist	2,000	4,000	8,000	16,000	32,000	64,000
10x10 Canny Edge Detection	48.7 ± 0.22	50.4 ± 0.13	53.4 ± 0.11	56.6 ± 0.06	58.8 ± 0.06	60.6 ± 0.03
10x10 Augmented Canny Edge Detection	50.6 ± 0.20	51.5 ± 0.21	53.2 ± 0.29	56.5 ± 0.21	59.9 ± 0.09	62.4 ± 0.06
5x5 Adaptive Gaussian Thresholding	57.9 ± 0.16	60.6 ± 0.17	63.0 ± 0.11	66.7 ± 0.10	68.2 ± 0.03	70.2 ± 0.03
5x5 Augmented Adaptive Gaussian Thresholding	57.8 ± 0.24	60.3 ± 0.15	63.3 ± 0.11	65.9 ± 0.04	69.0 ± 0.04	70.3 ± 0.02
10x10 Adaptive Mean Thresholding	57.8 ± 0.14	60.3 ± 0.12	62.3 ± 0.05	65.4 ± 0.08	67.2 ± 0.06	68.4 ± 0.13
10x10 Augmented Adaptive Mean Thresholding	59.1 ± 0.20	61.4 ± 0.16	63.4 ± 0.18	65.7 ± 0.06	68.3 ± 0.06	70.2 ± 0.06
10x10 Otsu's Thresholding	56.9 ± 0.13	58.5 ± 0.19	61.2 ± 0.17	62.9 ± 0.12	63.9 ± 0.23	65.0 ± 0.20
10x10 Augmented Otsu's Thresholding	58.5 ± 0.17	60.1 ± 0.21	61.5 ± 0.23	63.2 ± 0.20	65.2 ± 0.19	65.7 ± 0.31
3x3 Color Thermometers	63.6 ± 0.15	65.8 ± 0.42	68.4 ± 0.35	70.4 ± 0.26	72.3 ± 0.22	74.3 ± 0.15
4x4 Color Thermometers	64.6 ± 0.16	66.6 ± 0.33	68.9 ± 0.44	71.5 ± 0.30	73.6 ± 0.31	75.1 ± 0.32
5x5 Color Thermometers	64.5 ± 0.20	66.8 ± 0.17	69.1 ± 0.63	71.7 ± 0.27	73.7 ± 0.22	75.4 ± 0.09
3x3 Augmented Color Thermometers	64.1 ± 0.19	65.4 ± 0.52	67.5 ± 0.86	69.1 ± 1.26	70.5 ± 1.86	72.7 ± 0.96
4x4 Augmented Color Thermometers	65.4 ± 0.22	66.4 ± 0.46	68.5 ± 0.52	69.7 ± 1.49	71.6 ± 1.07	73.6 ± 1.44
5x5 Augmented Color Thermometers	65.1 ± 0.48	66.9 ± 0.46	68.6 ± 0.81	69.8 ± 1.87	71.7 ± 2.15	74.1 ± 1.35
Histogram of Oriented Gradients	64.2 ± 0.06	65.7 ± 0.04	66.9 ± 0.04	67.3 ± 0.03	67.5 ± 0.02	67.5 ± 0.02
Augmented Histogram of Oriented Gradients	64.8 ± 0.07	66.6 ± 0.08	67.5 ± 0.03	67.9 ± 0.02	68.4 ± 0.03	68.5 ± 0.02
3x3 Adaptive Color Thermometers	60.8 ± 0.10	63.2 ± 0.30	65.3 ± 0.14	68.1 ± 0.05	70.3 ± 0.06	71.7 ± 0.06
4x4 Adaptive Color Thermometers	61.3 ± 0.30	64.2 ± 0.22	66.9 ± 0.25	69.6 ± 0.07	71.5 ± 0.07	73.0 ± 0.05
5x5 Adaptive Color Thermometers	62.0 ± 0.25	65.1 ± 0.16	67.2 ± 0.14	69.3 ± 0.06	71.6 ± 0.06	73.3 ± 0.04
3x3 Augmented Adaptive Color Thermometers	61.3 ± 0.25	62.6 ± 0.22	64.8 ± 0.26	67.1 ± 0.53	68.9 ± 0.45	70.8 ± 0.65
4x4 Augmented Adaptive Color Thermometers	62.6 ± 0.23	64.1 ± 0.36	66.2 ± 0.21	68.4 ± 0.33	70.3 ± 0.75	72.9 ± 0.33
5x5 Augmented Adaptive Color Thermometers	62.9 ± 0.21	64.6 ± 0.34	66.4 ± 0.39	68.5 ± 0.65	71.1 ± 0.40	72.9 ± 0.38
Added Together in a TM Composite	79.5 ± 0.04	80.6 ± 0.02	81.5 ± 0.02	82.2 ± 0.02	82.7 ± 0.02	82.8 ± 0.01



Y Grønningsæter, HS Smørvik, OC Granmo. An Optimized Toolbox for Advanced Image Processing with Tsetlin Machine Composites, *2024 International Symposium on the Tsetlin Machine (ISTM)*, Pittsburgh, US, 2024.



Hypervectors

Sparse binary hypervectors

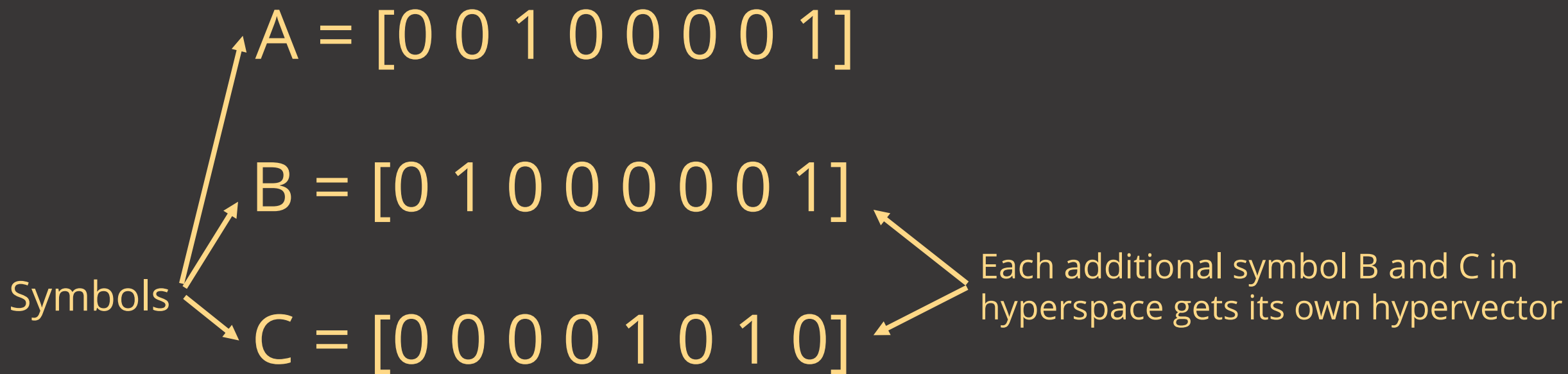
Two randomly set bits defines the symbol A in hyperspace

$A = [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1]$

Symbol

Rachkovskij DA (2001) Representation and processing of structures with binary sparse distributed codes. IEEE Trans Knowl Data Eng 13(2):261–276.

Sparse binary hypervectors



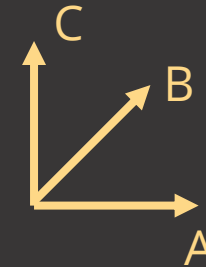
A point in hyperspace is set by bundling

Bundling: We create a bundle with bitwise OR (|)

$$A = [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1]$$

$$C = [0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0]$$

$$A \oplus C = A \mid C = [0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1]$$



Binding creates complex structure

Binding: We bind with bitwise shift (>>)

$$A = [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1]$$

$$A_{\text{shift}} = 2$$

Randomly set shifting
value



$$B = [0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1]$$

$$B_{\text{shift}} = 5$$

$$C = [0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0]$$

$$C_{\text{shift}} = 3$$

Binding creates complex structure

Binding: We bind with bitwise shift (>>)

$$A = [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1] \quad A_{\text{shift}} = 2$$

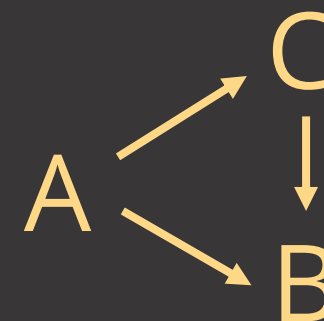
$$C = [0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0] \quad C_{\text{shift}} = 3$$

$$A \otimes C = [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1] \gg C_{\text{shift}} =$$

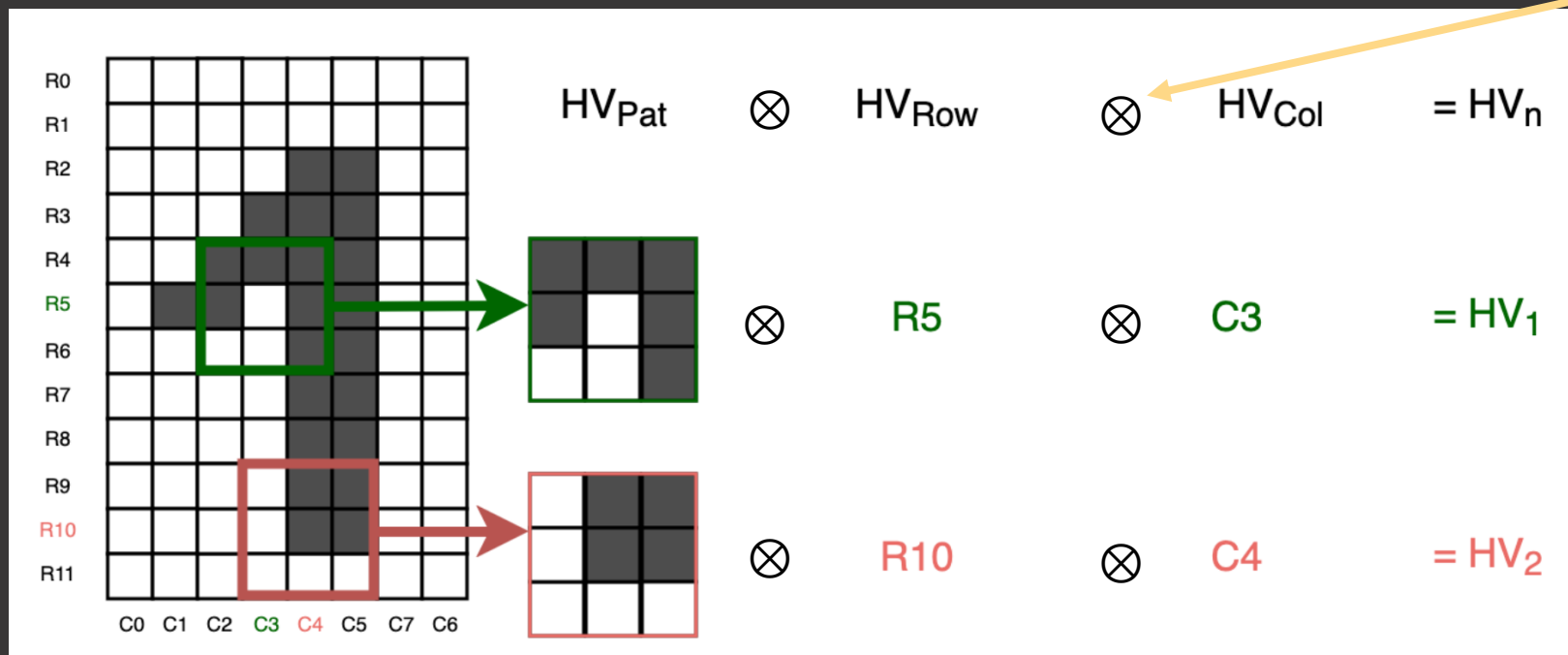
$$[0 \ 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0]$$

Binding and bundling a directed graph

$$(A \otimes C) \oplus (A \otimes B) \oplus (C \otimes B)$$



Binding and bundling an image



Binding

Bundle: $HV_1 \oplus HV_2$

Bundling



V. Halenka *et al.*, "Exploring Effects of Hyperdimensional Vectors for Tsetlin Machines," *2024 International Symposium on the Tsetlin Machine (ISTM)*, Pittsburgh, US, 2024.

Decisions

Yes/No

Yes/No

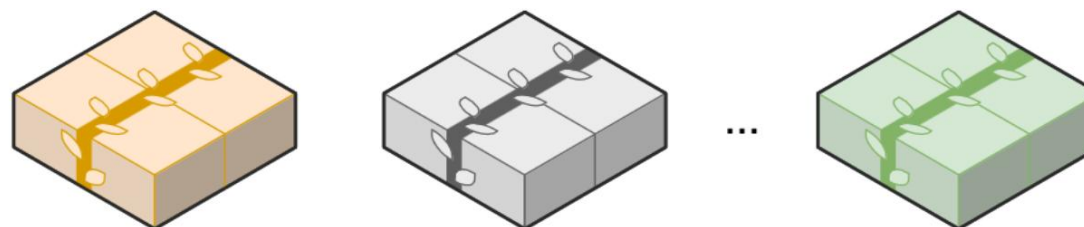
Hyper Clauses



Hyper Automata

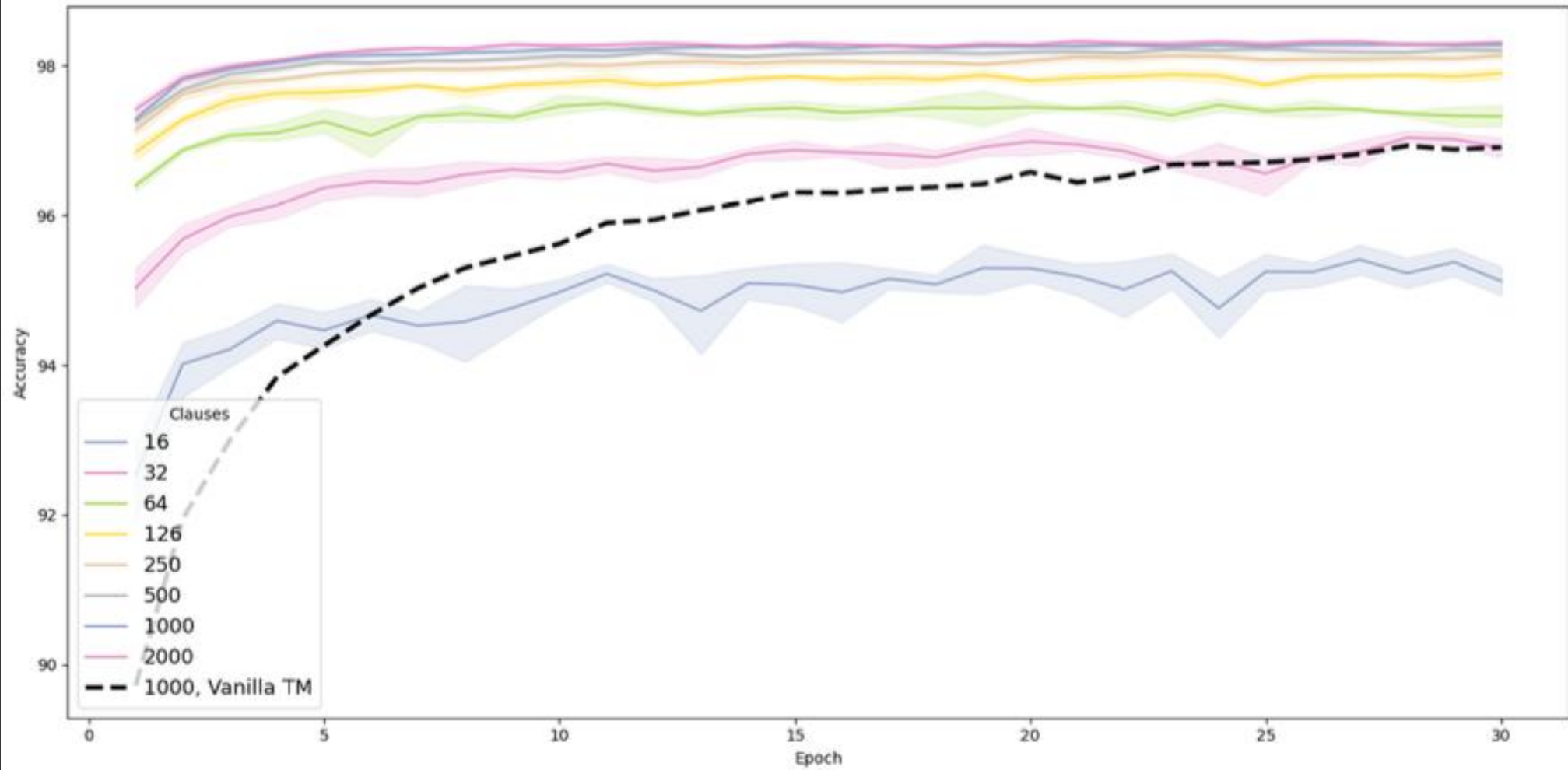


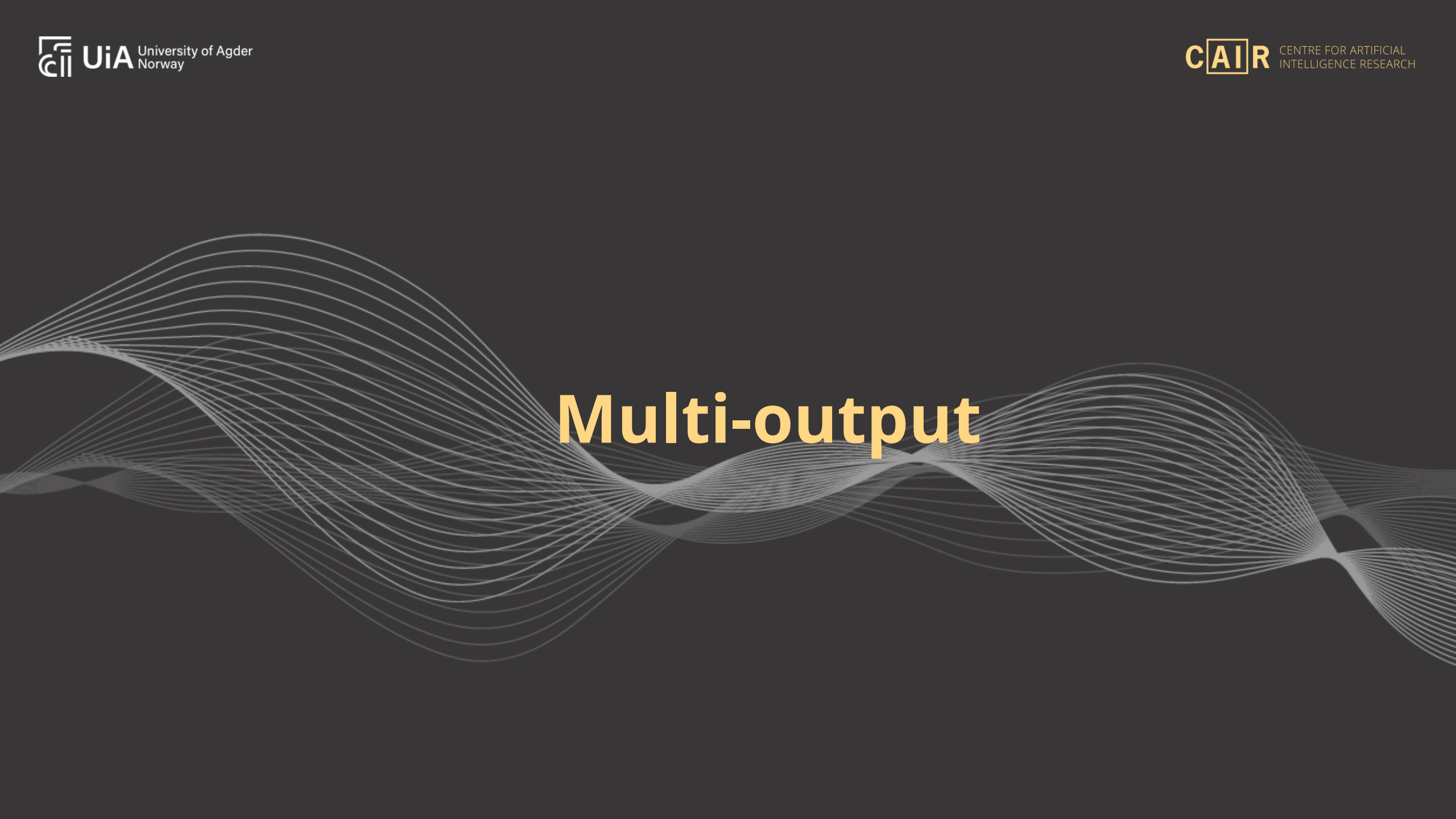
**Hypervector
w/Bundling &
Binding**



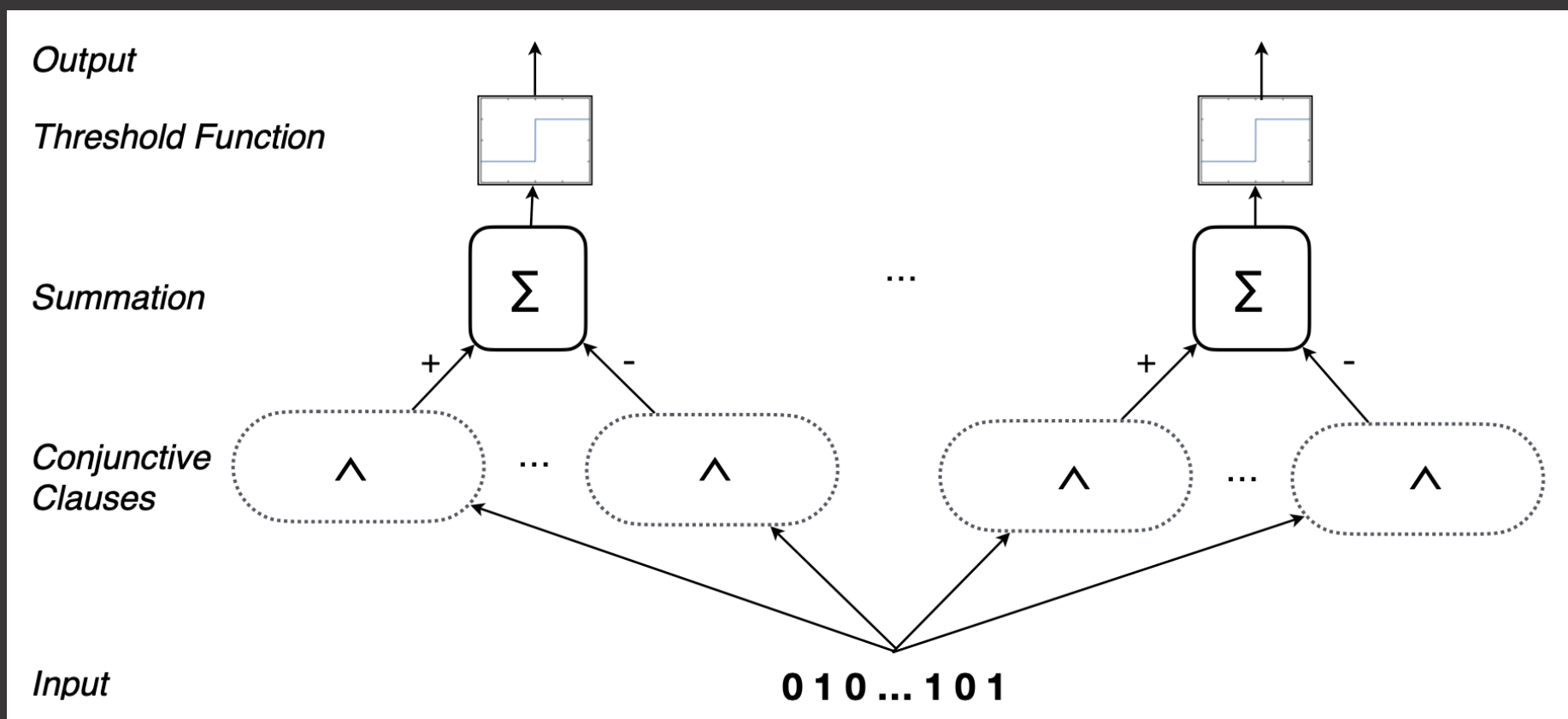
Input

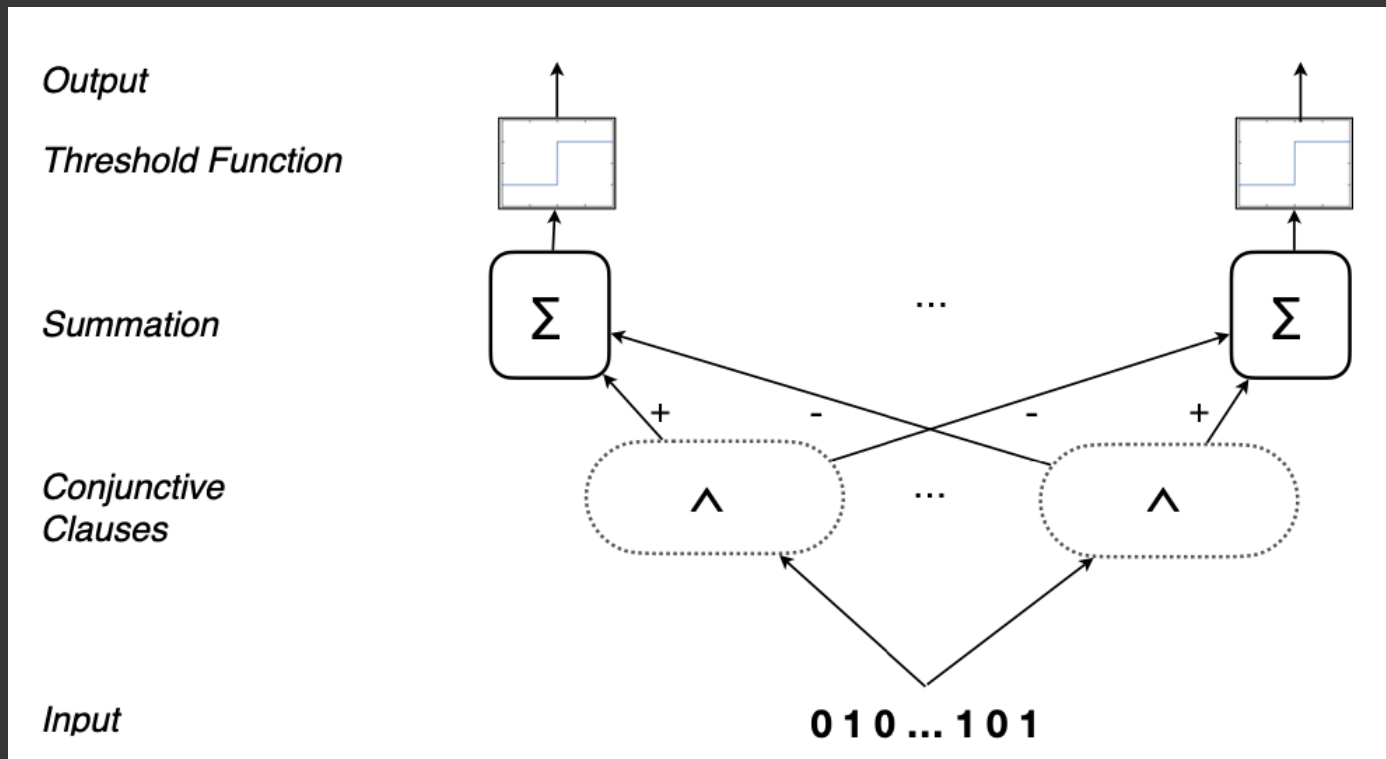




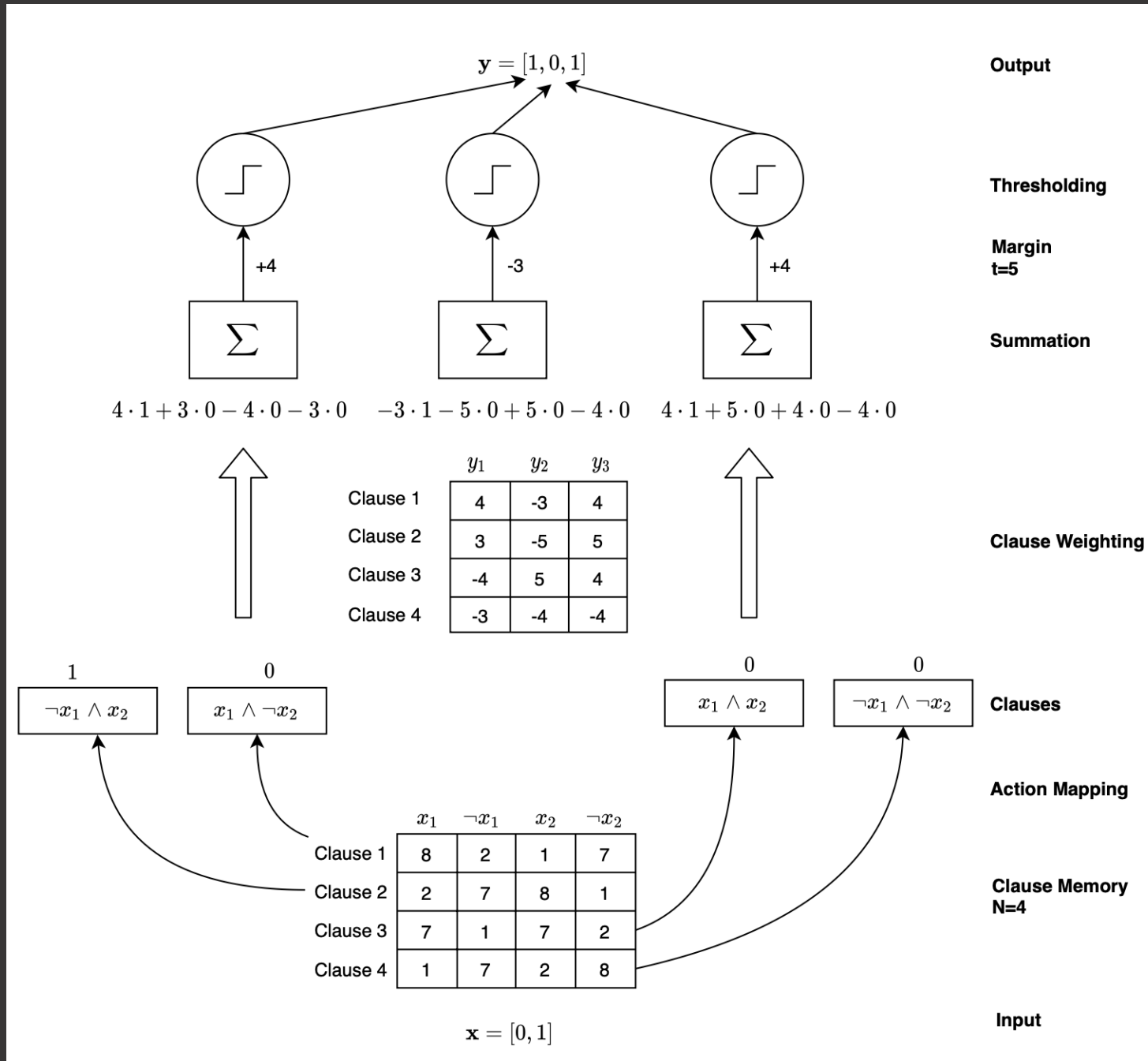


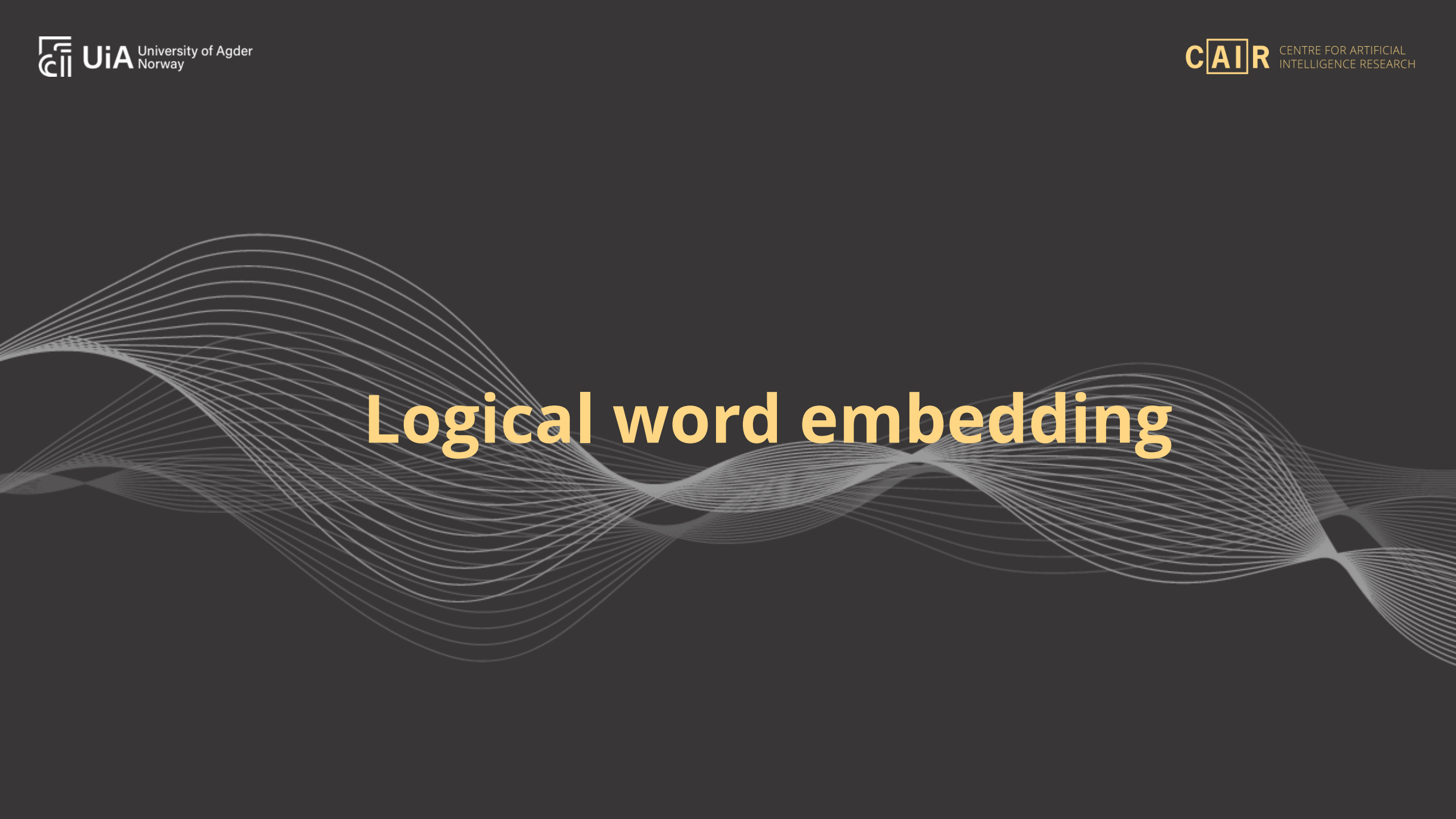
Multi-output





Sondre Glimsdal and Ole-Christoffer
 Granmo (2021). Coalesced Multi-
 Output Tsetlin Machines with Clause
 Sharing. *arXiv:2108.07594*





Logical word embedding

Logical word embedding

Queen: *"the female ruler of an independent state, especially one who inherits the position by right of birth."*



Bhattarai, Bimal; Granmo, Ole-Christoffer; Jiao, Lei; Yadav, Rohan; Sharma, Jivitesh (2023-01-03). "Tsetlin Machine Embedding: Representing Words Using Logical Expressions"

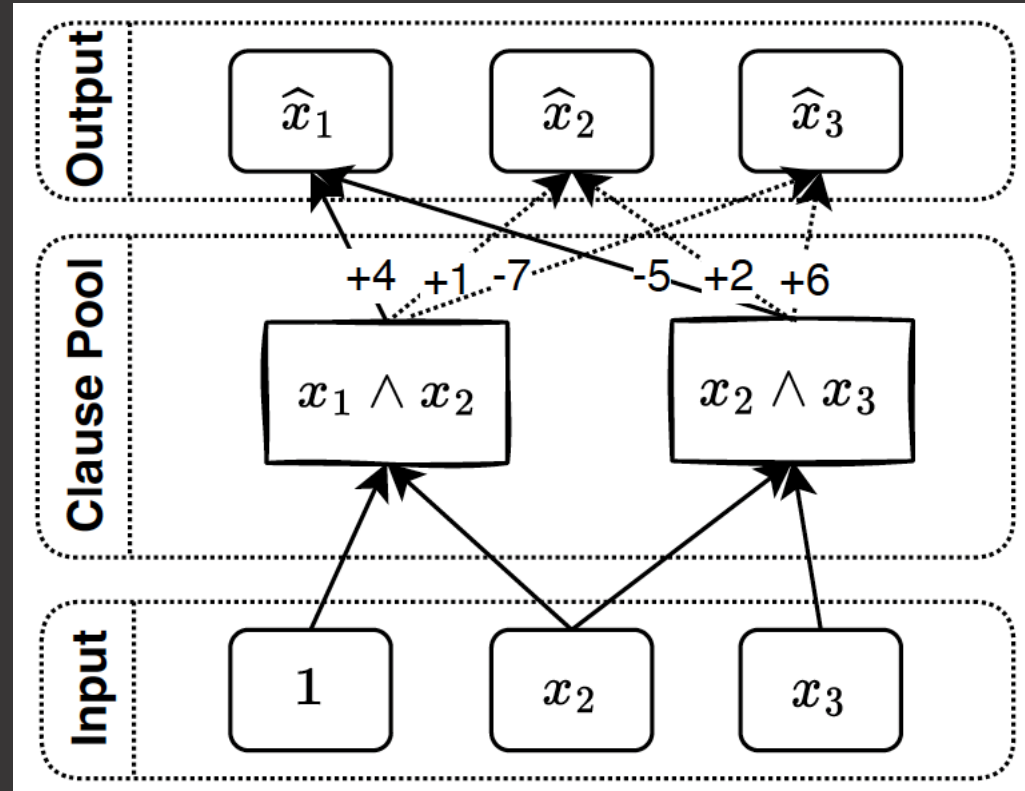
Neural network word embeddings

Embedding "queen":

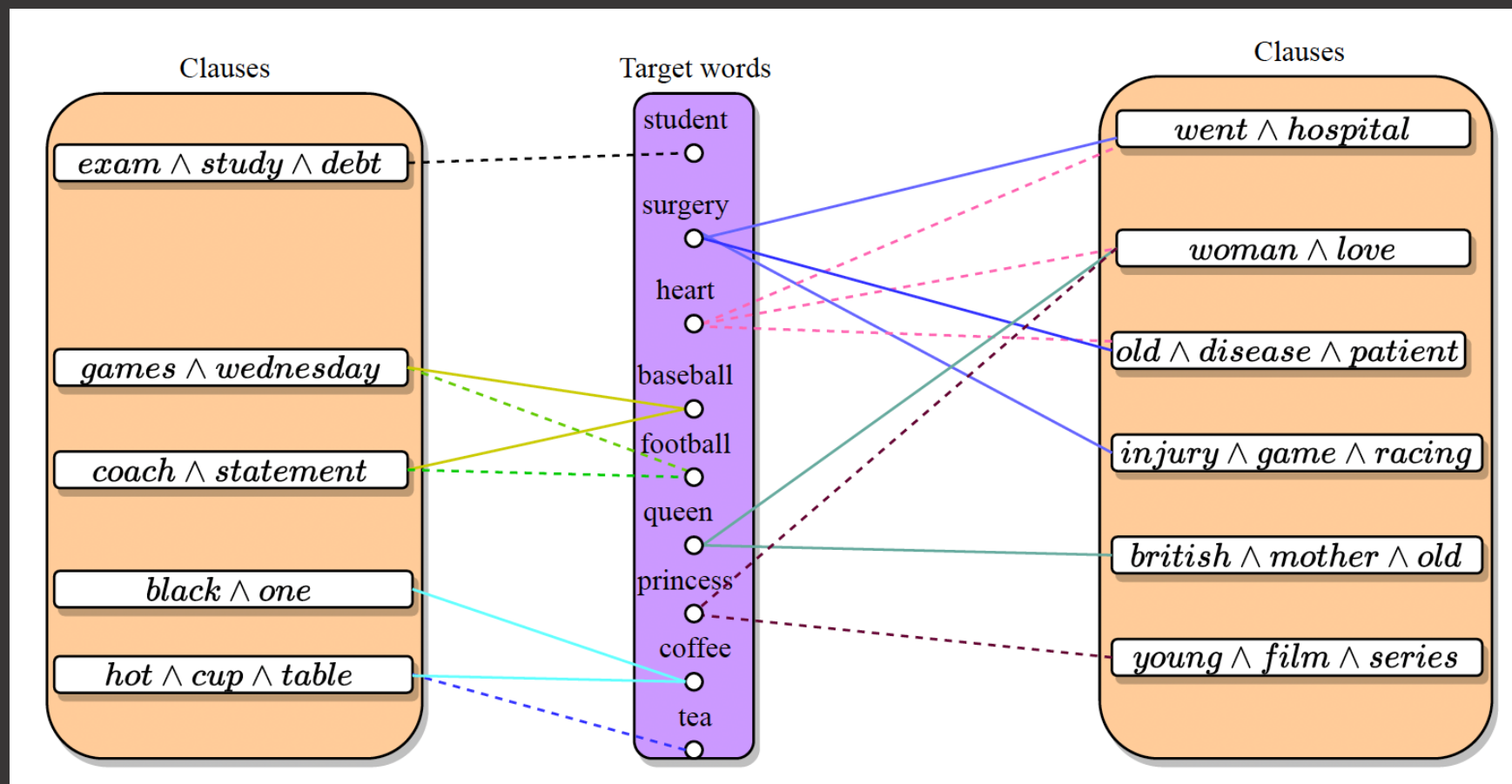
```

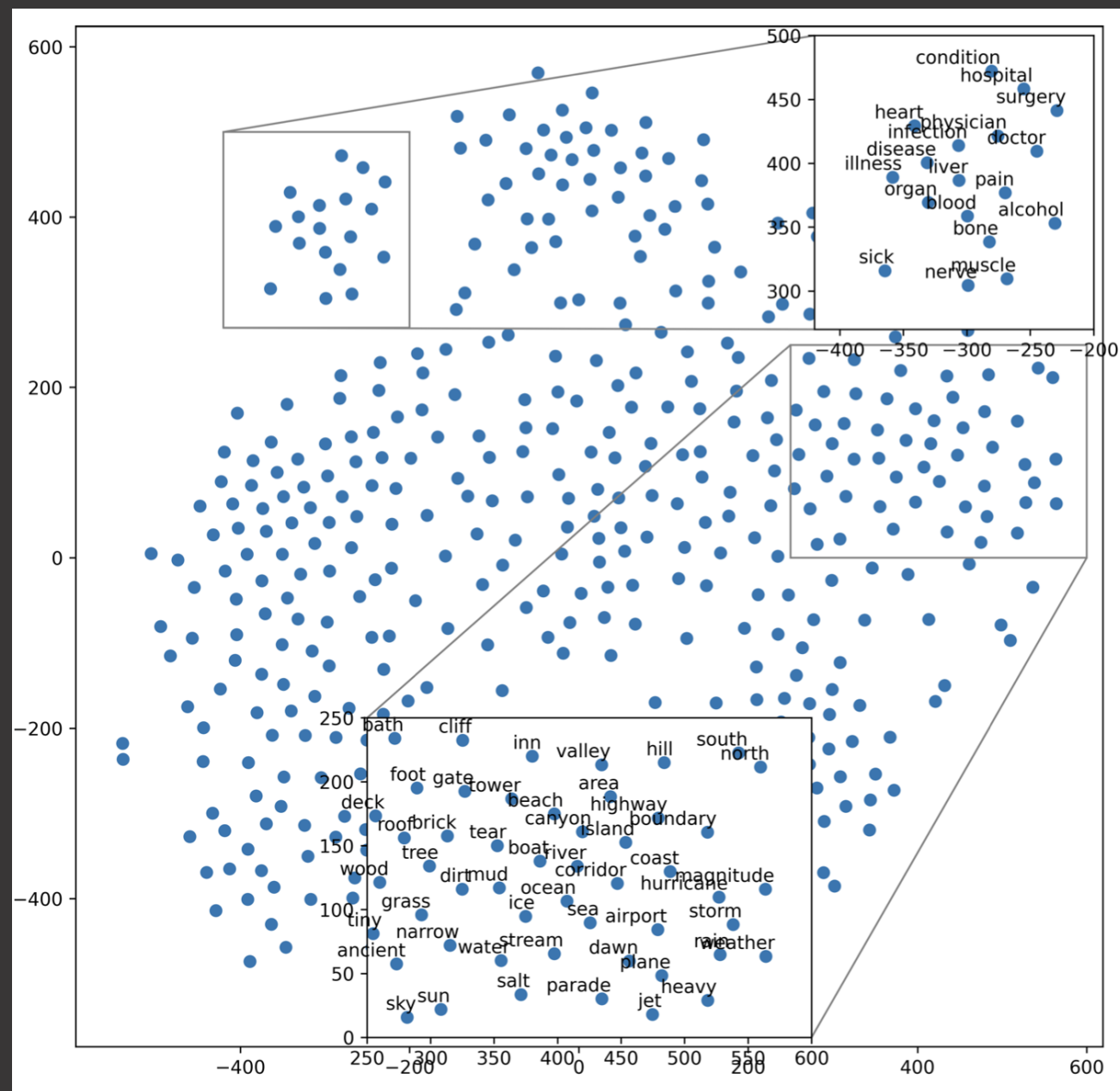
array([ -0.32107702,  0.02634325,  0.5982603 , -0.6479276 ,  0.3632543 ,
        -0.30940267,  0.11201022, -1.4546527 , -0.6258229 ,  0.06379458,
        -0.5263066 , -0.14645165, -0.08704363, -0.45985267, -0.02743918,
        -0.14499499,  1.0008129 ,  0.00337885, -0.08546805,  0.728745 ,
        -0.5122128 , -0.03007157, -0.04415359,  0.09485263,  0.8781621 ,
        -0.9580598 , -0.7981698 , -0.4311728 ,  0.67374474, -0.8651508 ,
         0.5282616 ,  0.04314229,  0.82111883, -0.93143344,  0.17256826,
         0.3960089 ,  0.15104313,  0.620224 ,  0.08957284, -0.33669117,
        -0.6600646 ,  0.5423459 , -0.41448766, -1.0866758 , -0.41825002,
         0.93874204, -0.4660616 , -0.2024687 ,  0.26992255,  0.10713856,
        -0.47868192,  0.11337369, -0.43584147,  0.28840148, -0.457471 ,
        -1.4268371 , -0.04075855, -0.3103302 , -0.00450425,  0.7783772 ,
         0.50324357, -0.43334493,  0.44846568, -1.1994292 ,  0.12398864,
         0.48461846, -0.6658262 ,  0.77683586, -0.4329836 ,  0.90014684,
         0.29278183, -0.7190529 ,  1.0714668 , -0.33908844, -0.02098005,
        -0.02108962, -0.60200346, -0.15406518,  0.03903955,  0.15439767,
        -0.7829167 , -0.0208698 ,  0.38933557, -0.52256554,  0.27805656,
        -0.00311717,  0.355087 , -0.4900707 ,  0.39375502, -0.31120643,
         1.0033151 ,  0.19191714,  0.47006622, -0.33764473,  0.9833659 ,
        -0.20627594,  0.4717329 ,  0.6750194 , -0.1344328 ,  0.45001876,
    
```

Logical word embeddings with Tsetlin machines



Logical word embeddings with Tsetlin machines

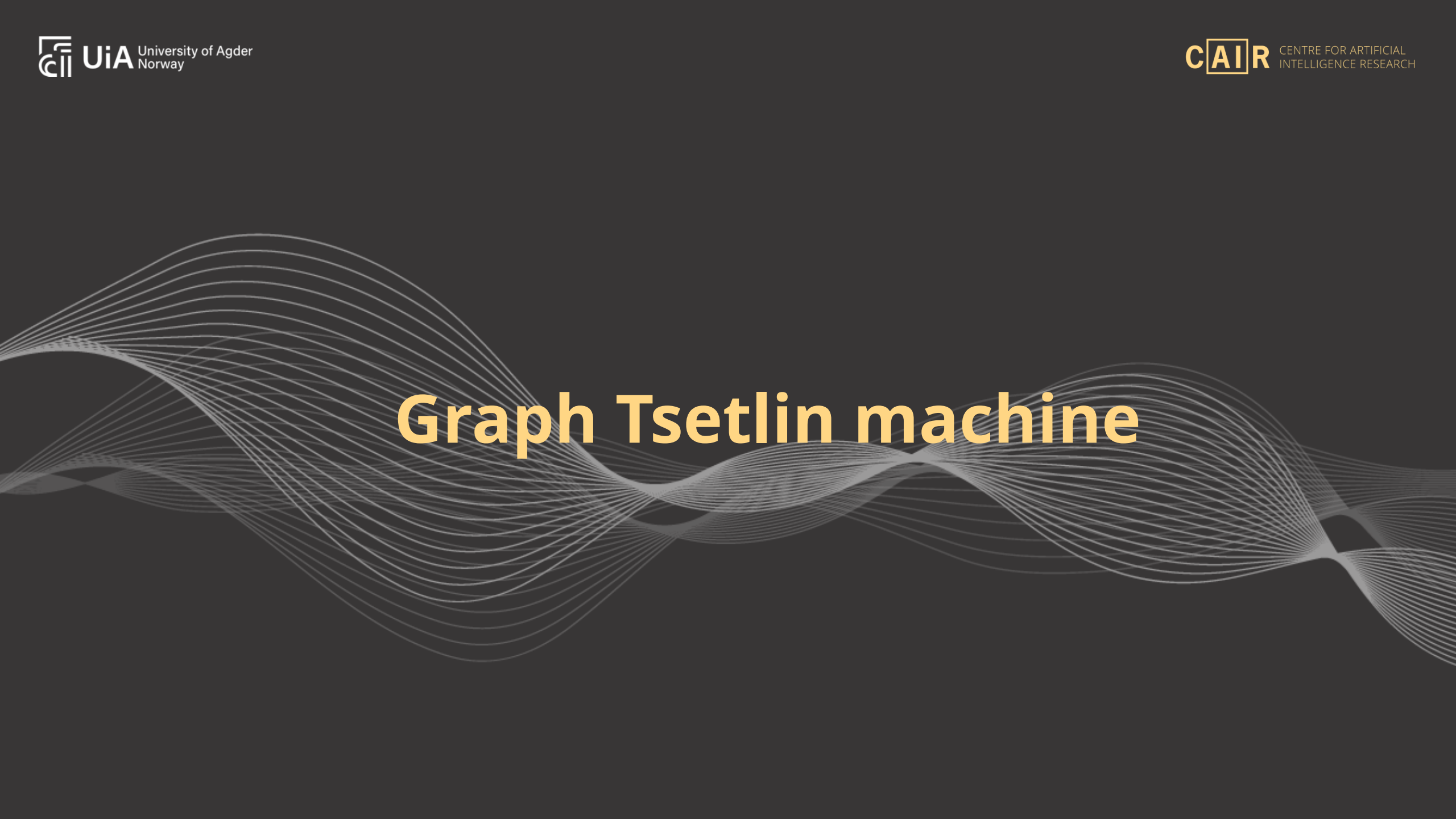




<https://github.com/cair/tmu/blob/main/examples/autoencoder/IMDbWordEmbeddingDemo.py>

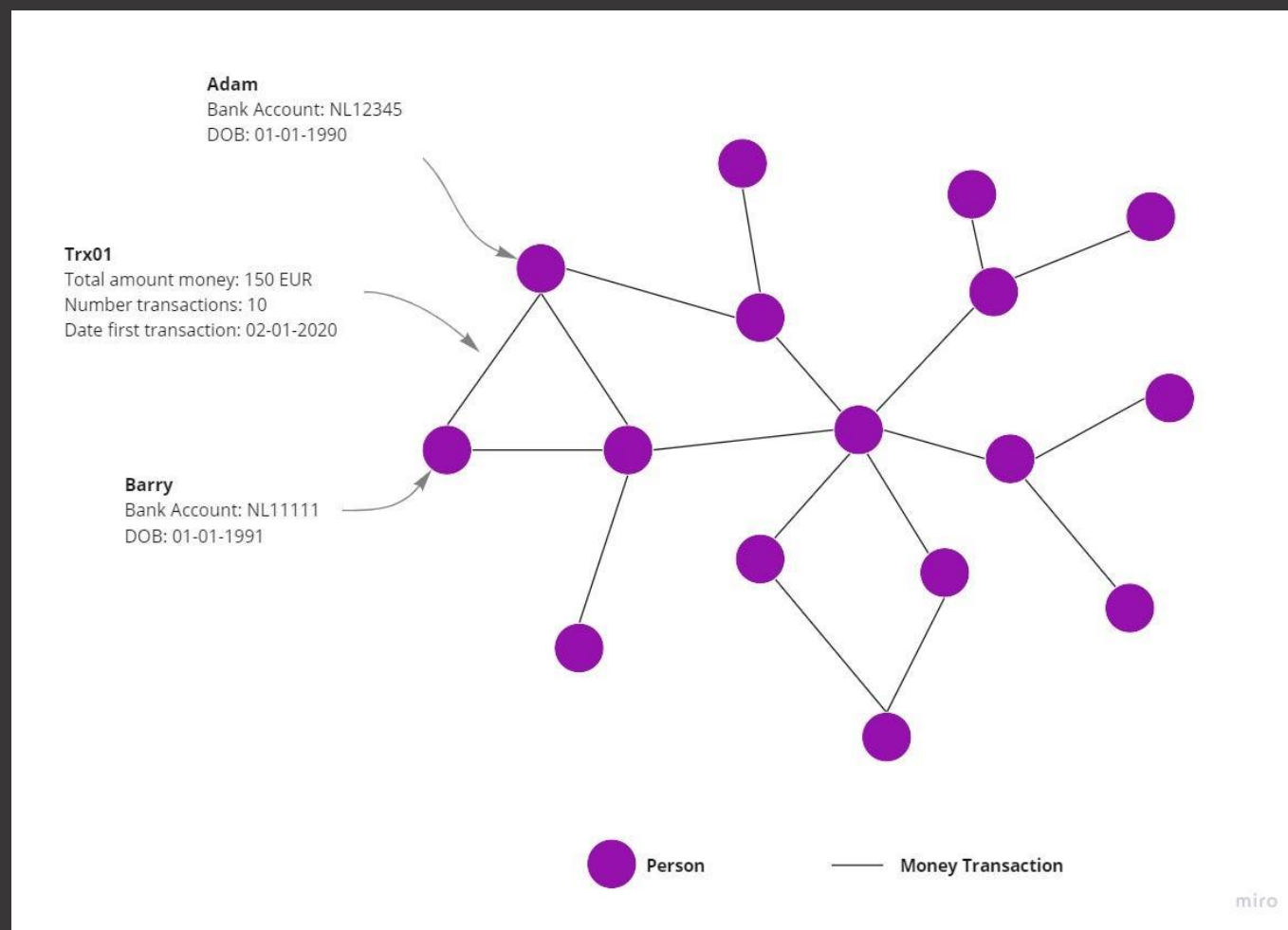
```
tm = TMAutoEncoder(courses, T, s, output_active, max_included_literals=3, accumulation=accumulation, feature_negation=False)

print("\nAccuracy Over 40 Epochs:")
for e in range(40):
    start_training = time()
    tm.fit(X_train, number_of_examples=number_of_examples)
    stop_training = time()
```

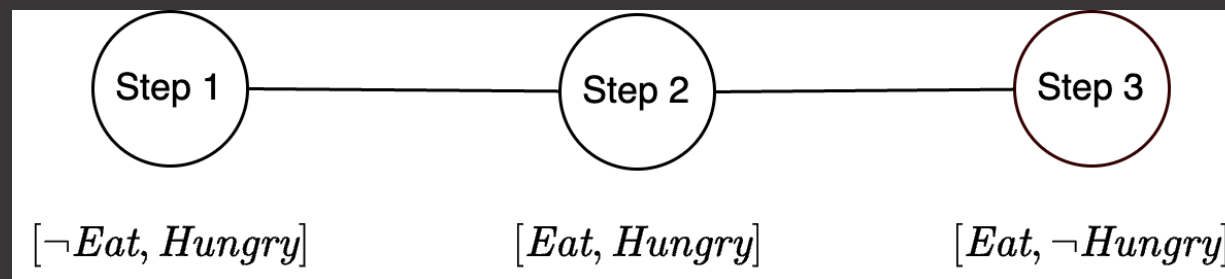


Graph Tsetlin machine

Example: money transaction analysis



Example: temporal logic

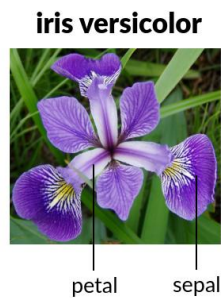


I am *always* hungry

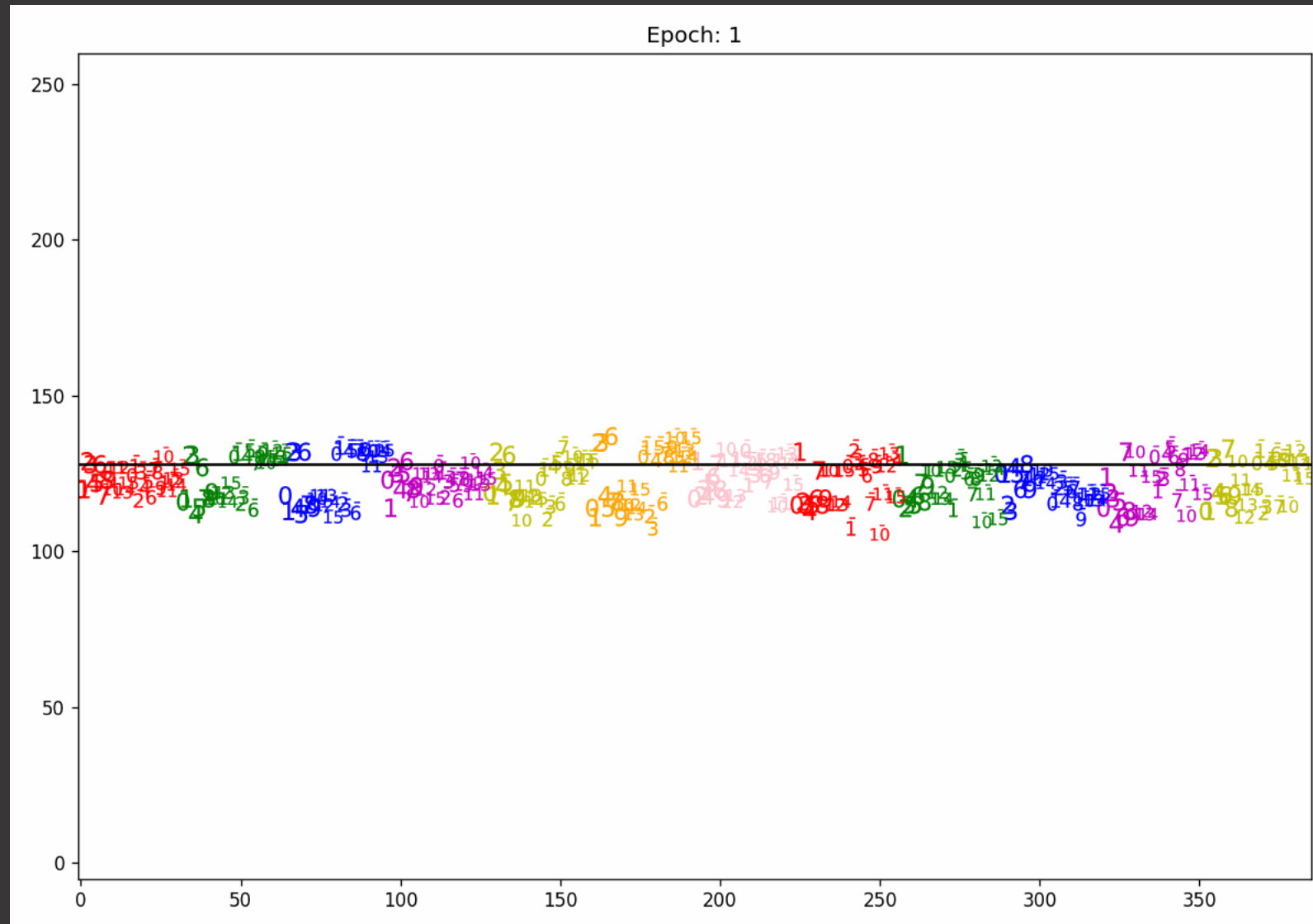
I will *eventually* be hungry

I will be hungry *until* I eat something

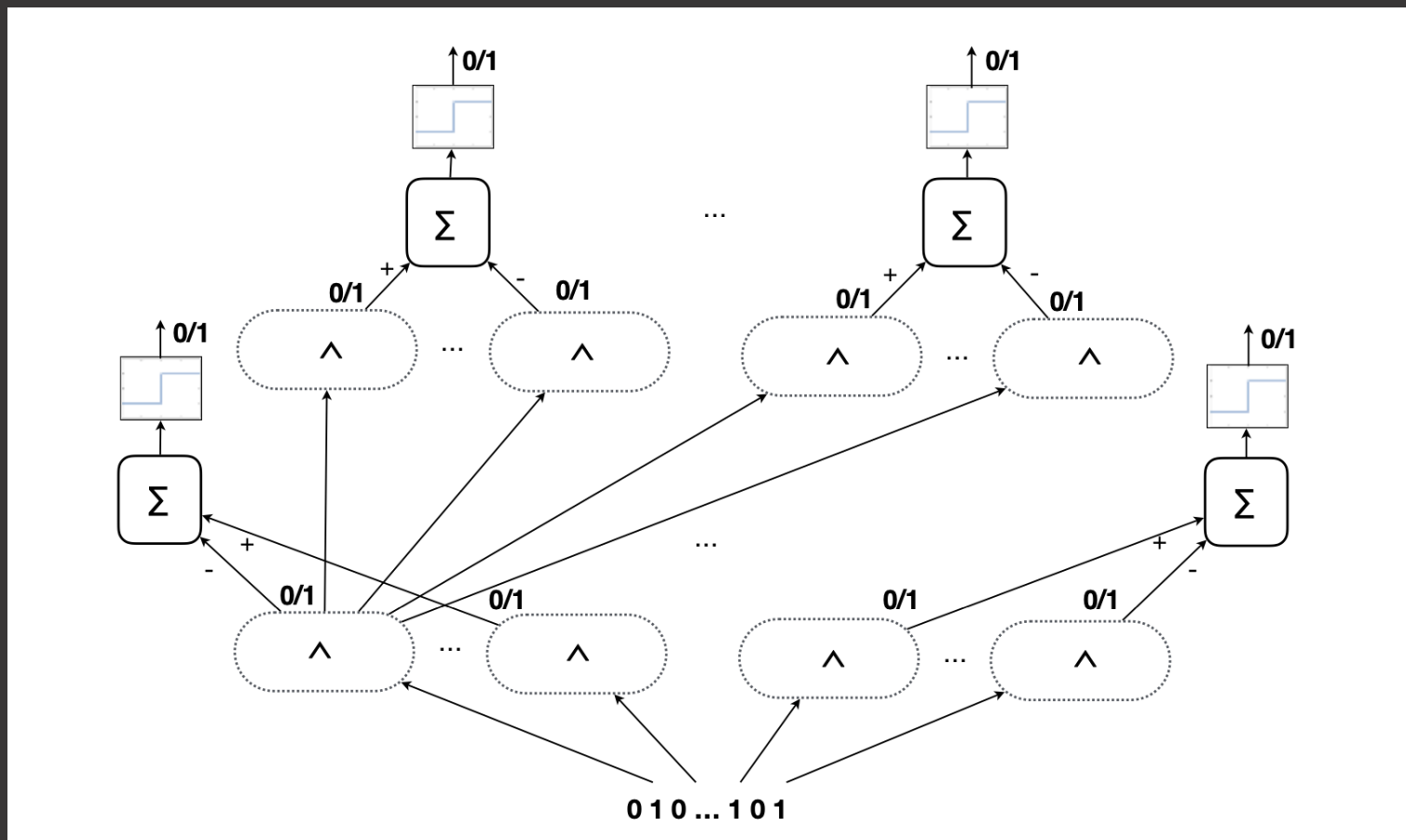
Coordinated learning



BY PRAMOD SAHU

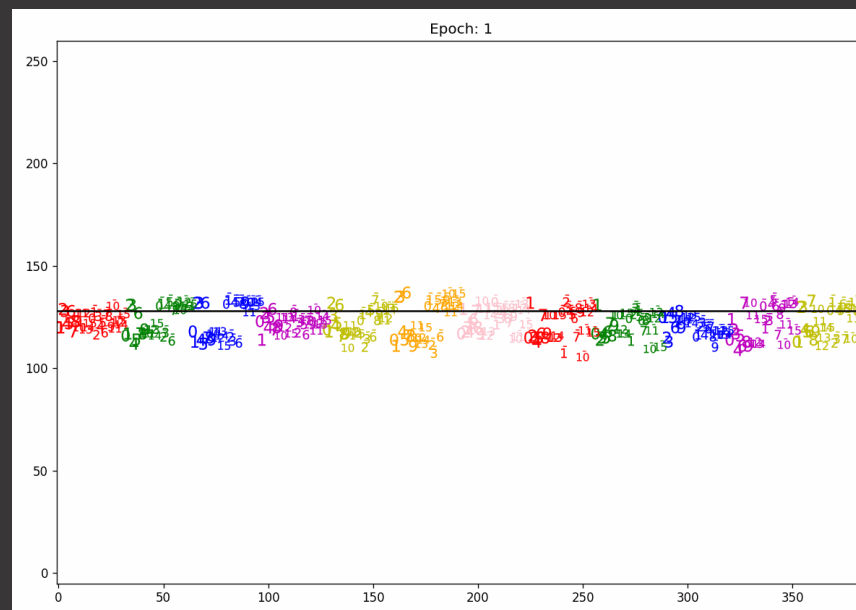


Fully connected deep Tsetlin machine

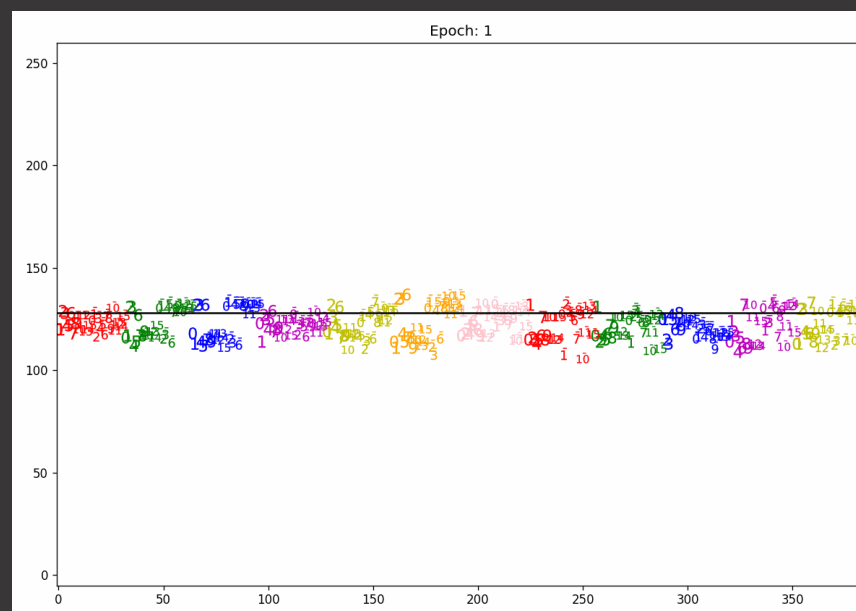


How to coordinate two layers?

Layer 2



Layer 1



iris setosa



petal sepal

iris versicolor



petal sepal

iris virginica



petal sepal

BY PRAMOD SAHU

Bi-directional interconnections



petal sepal

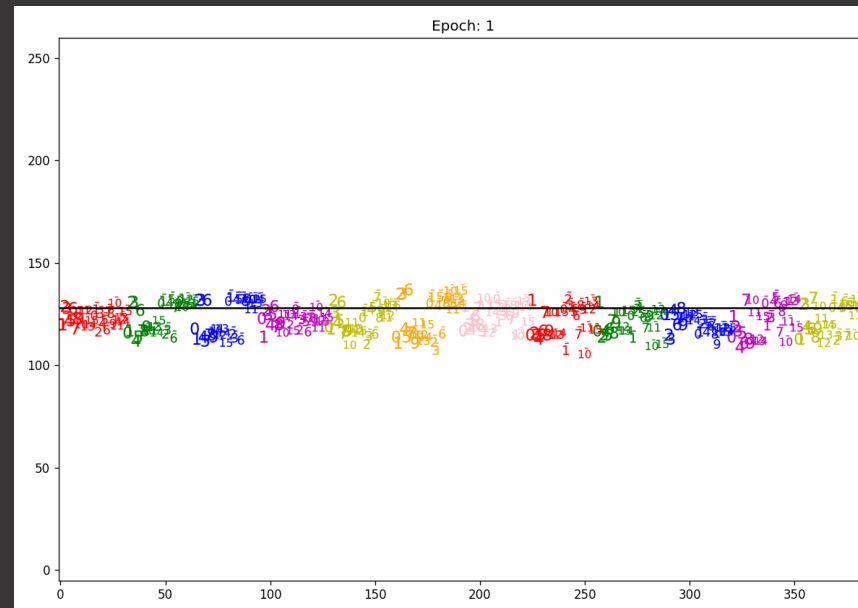


petal sepal



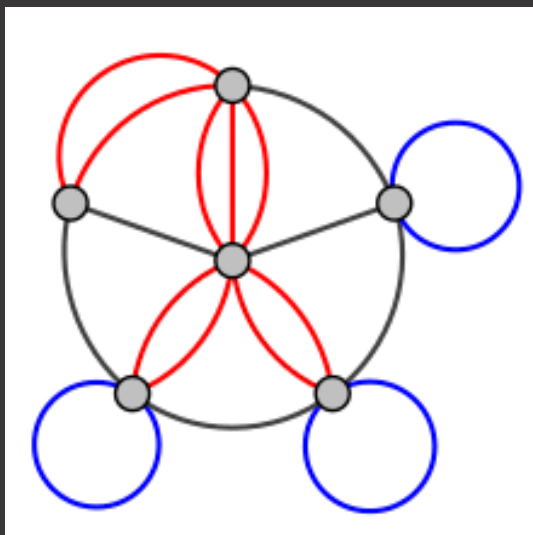
petal sepal

BY PRAMOD SAHU



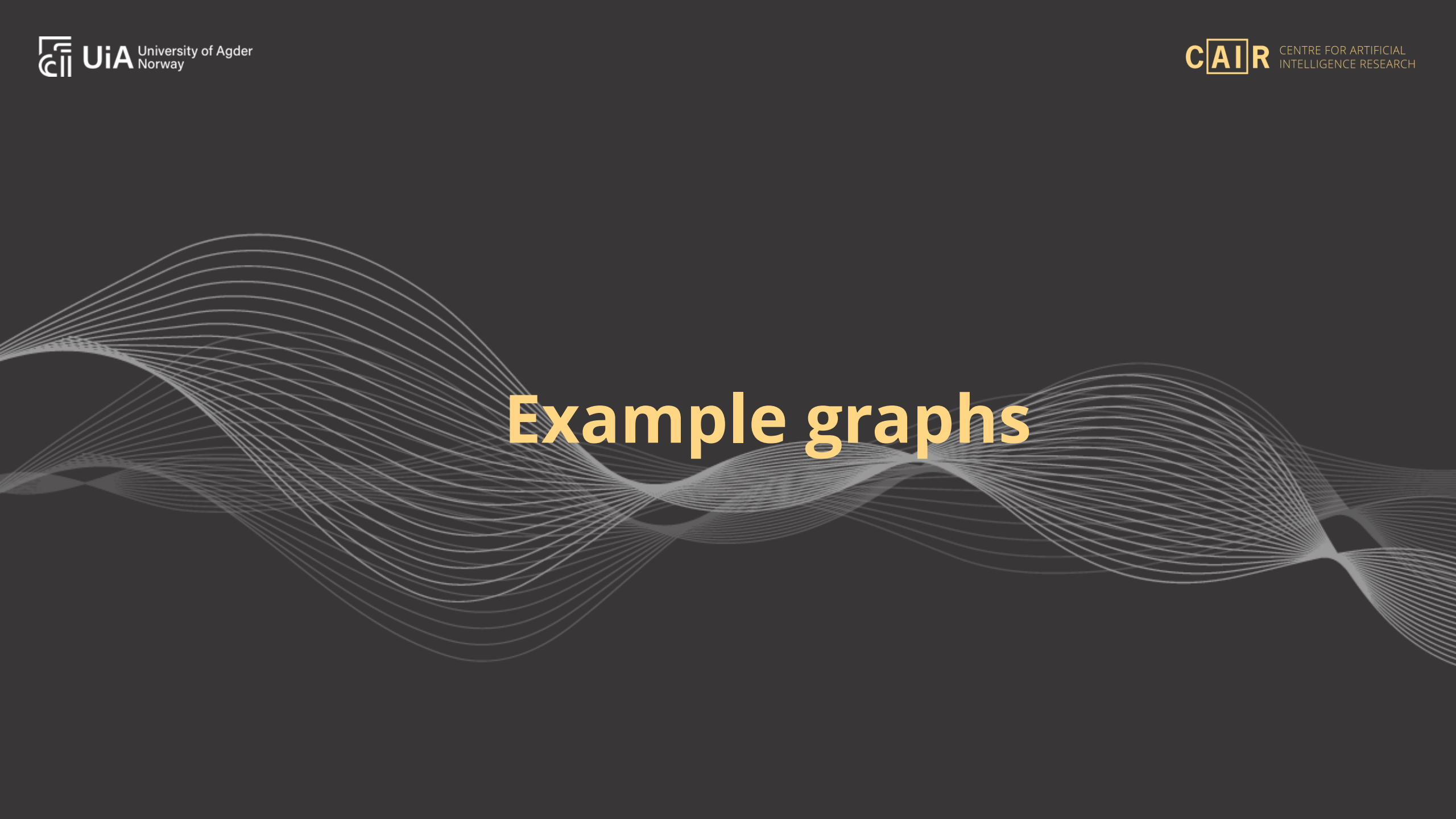
Layer 1

Tsetlin machine for logical learning and reasoning with graphs



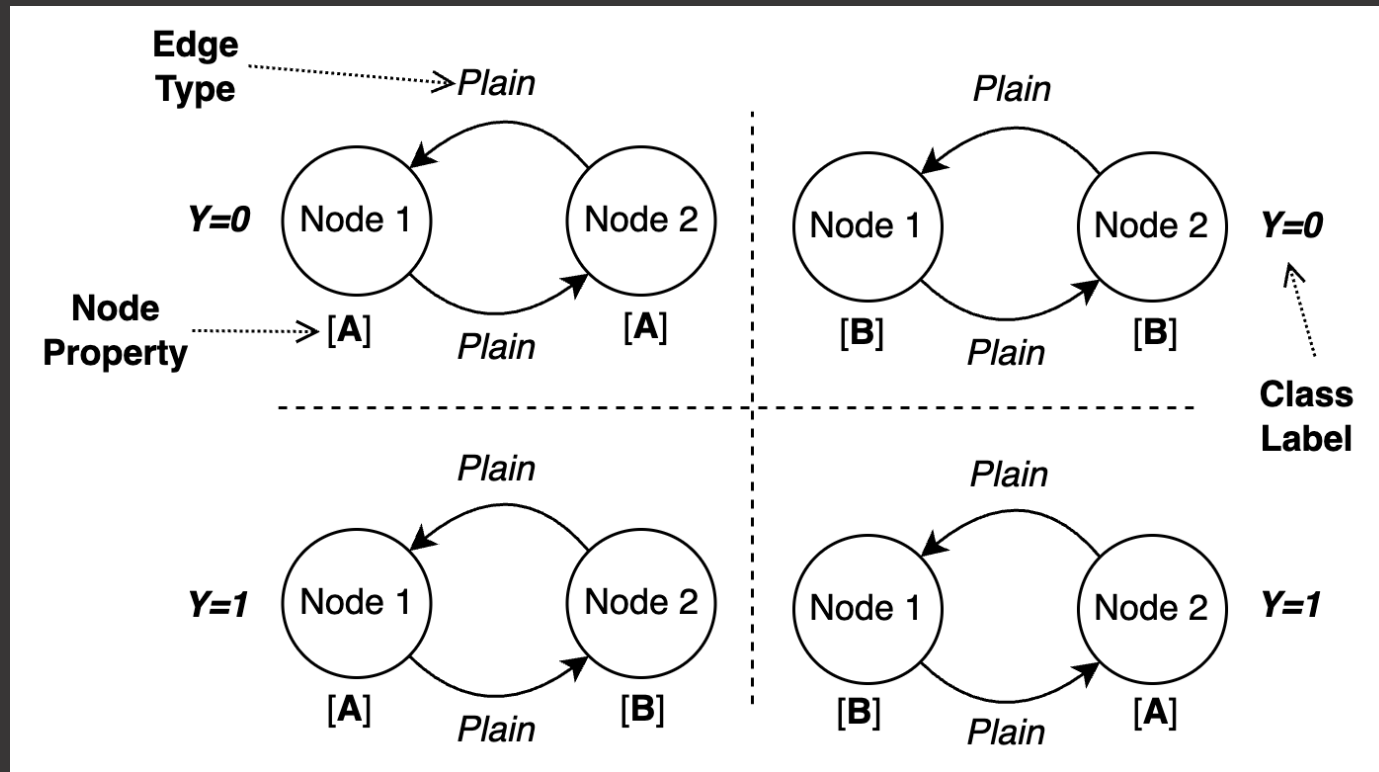
- Processes directed and labeled multigraphs
- Vector symbolic node properties and edge types
- Nested (deep) clauses
- Arbitrarily sized inputs
- Incorporates Vanilla, Multiclass, Convolutional, and Coalesced Tsetlin Machines
- Rewritten faster CUDA kernels

<https://github.com/cair/GraphTsetlinMachine>

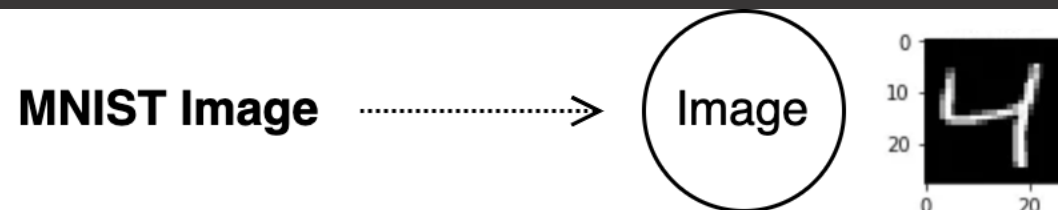


Example graphs

Noisy XOR

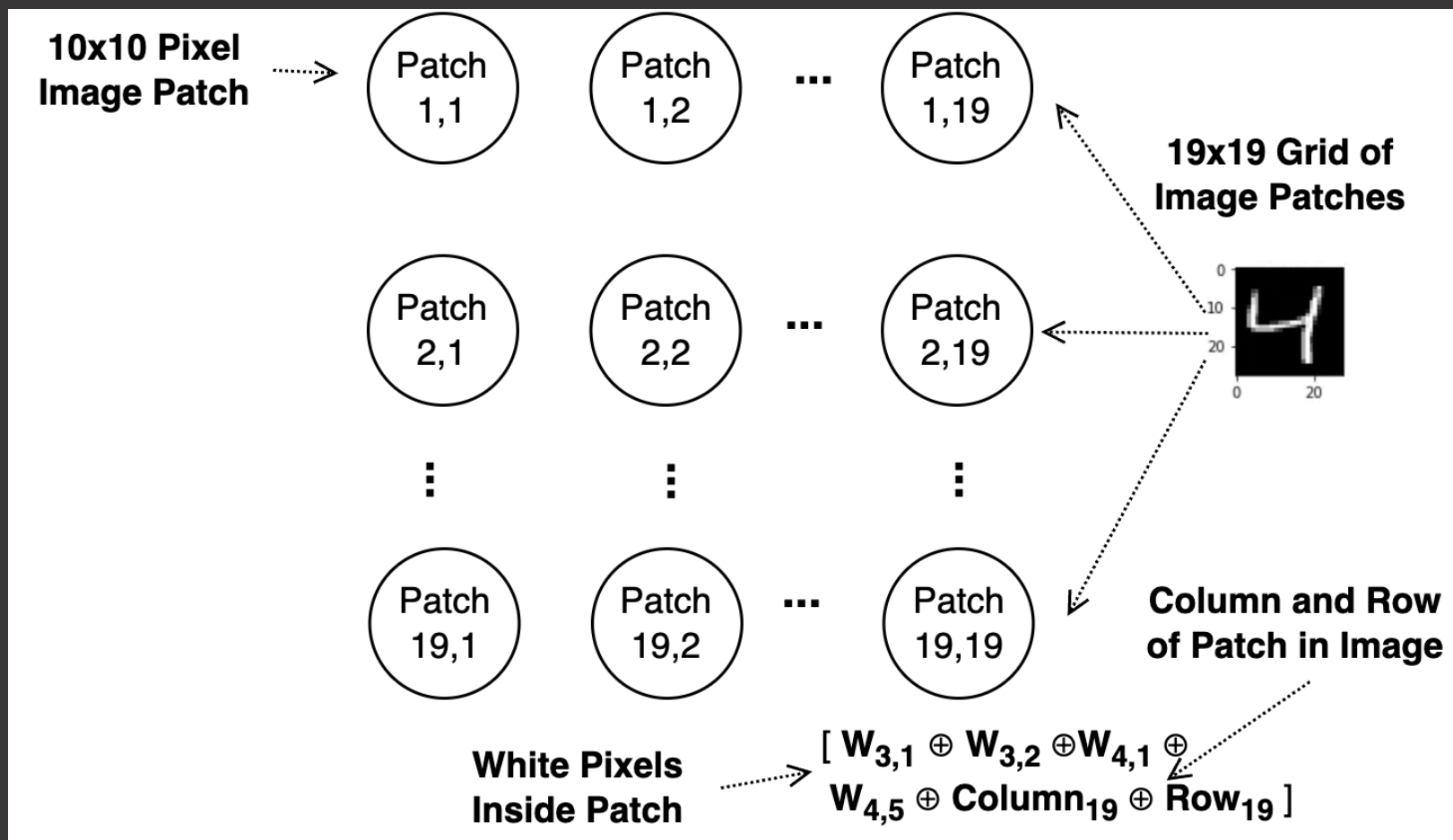


Vanilla MNIST

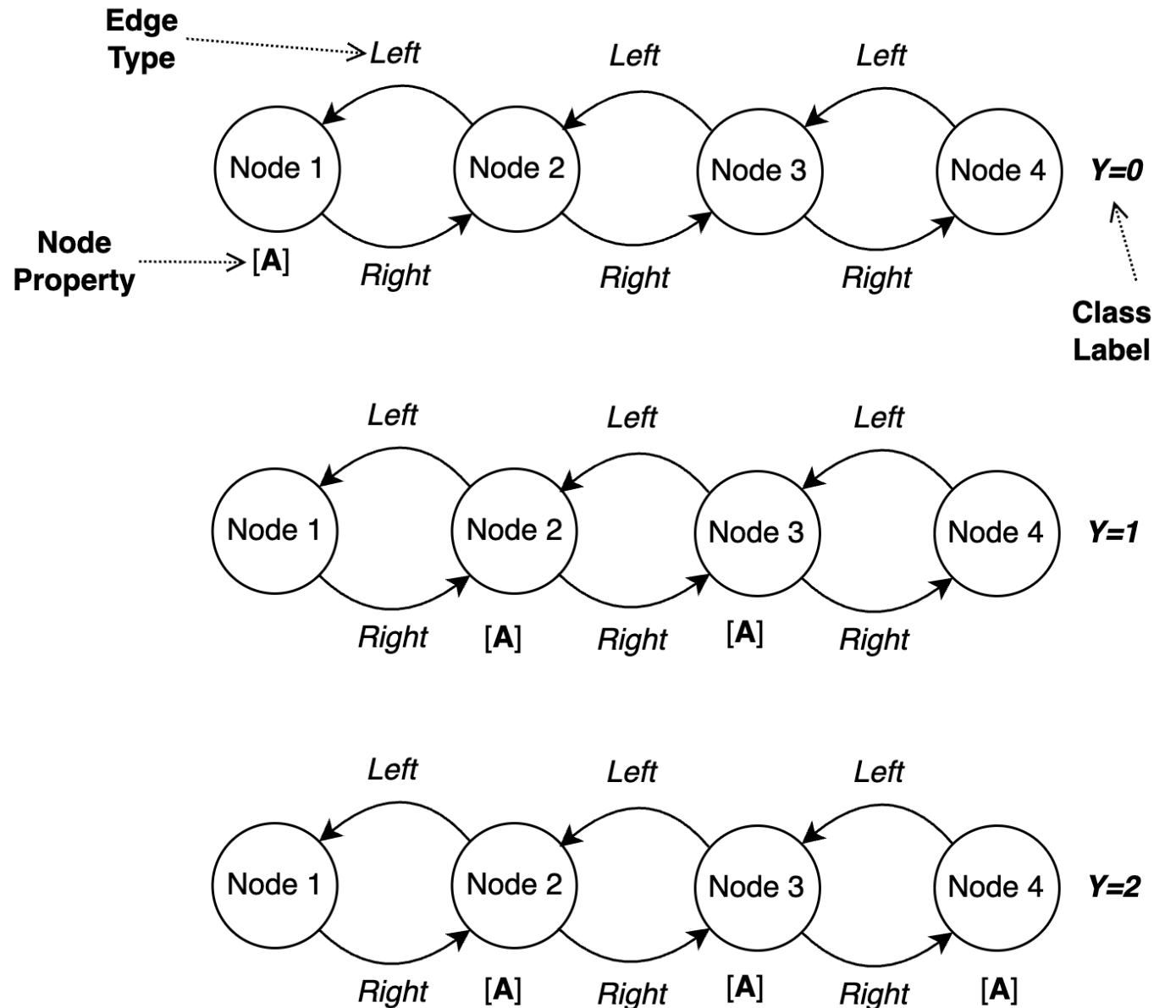


White Pixels>
$$\begin{aligned}
 &[W_{7,13} \oplus W_{7,14} \oplus W_{7,15} \\
 &\quad \oplus \dots \oplus \\
 &W_{25,16} \oplus W_{25,17} \oplus W_{26,16}]
 \end{aligned}$$

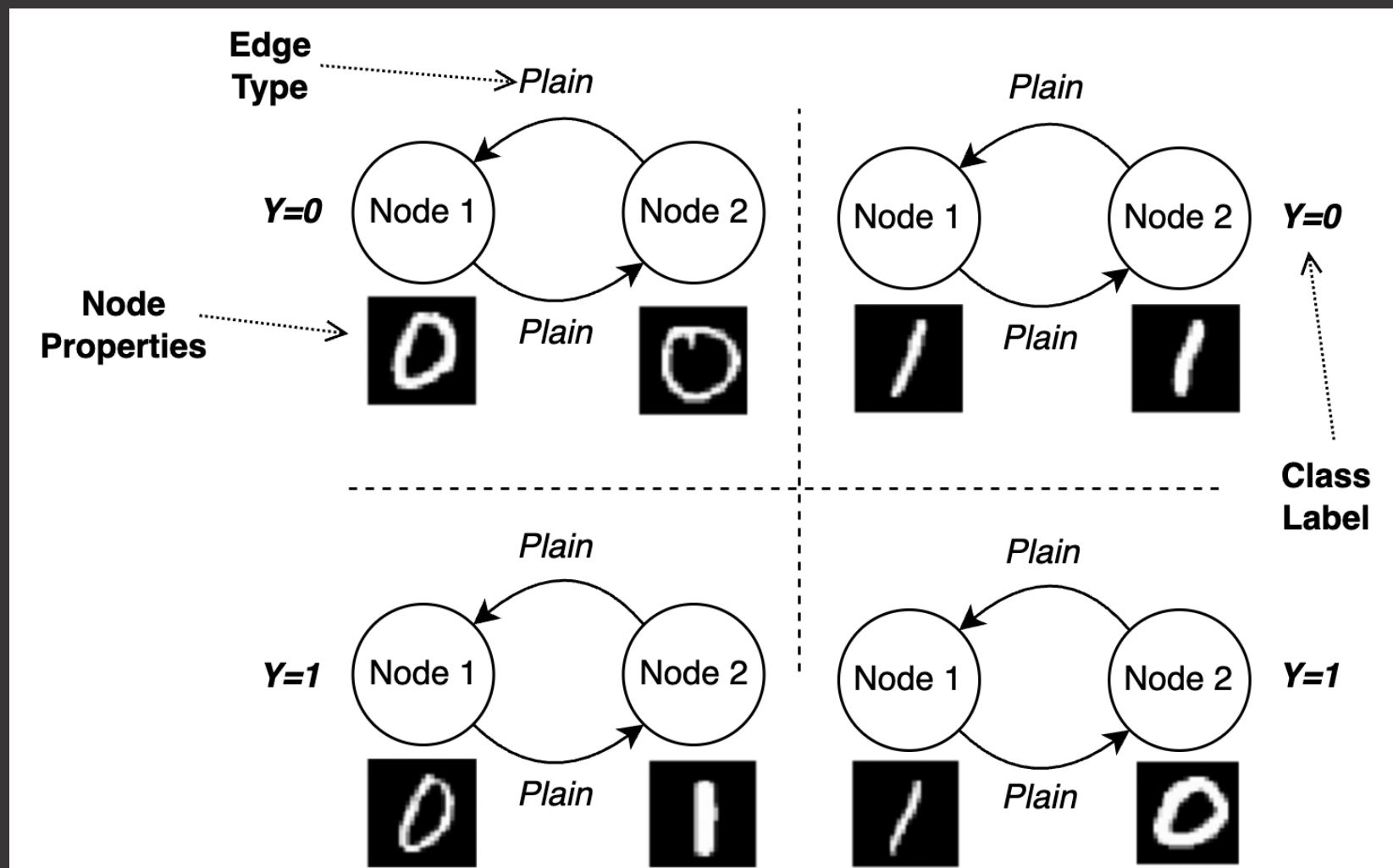
Convolutional MNIST



Sequence classification



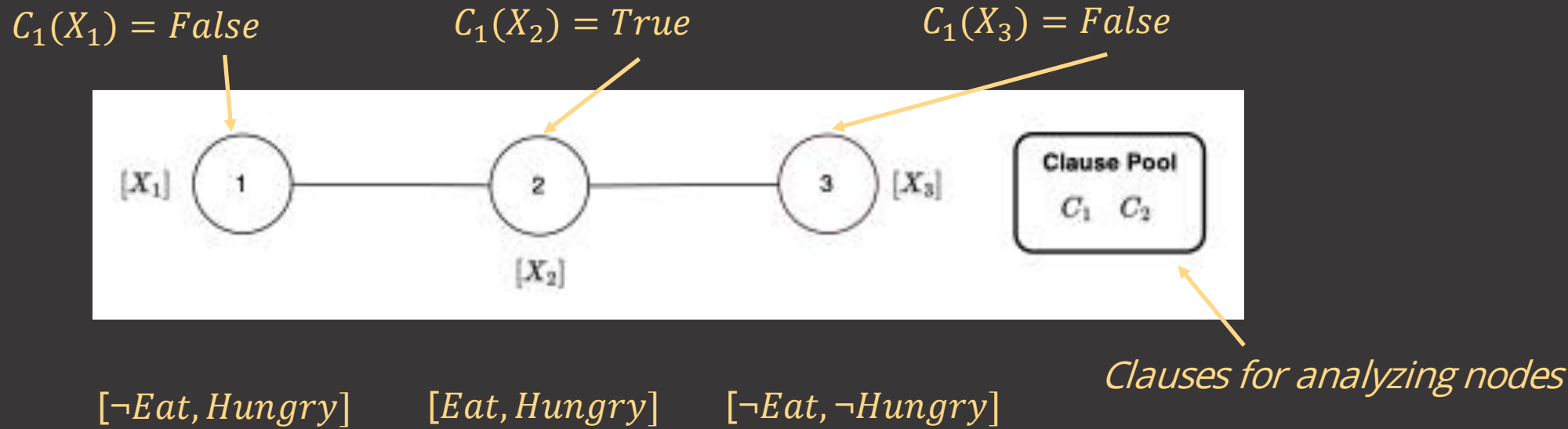
Noisy XOR w/MNIST



Nested clauses



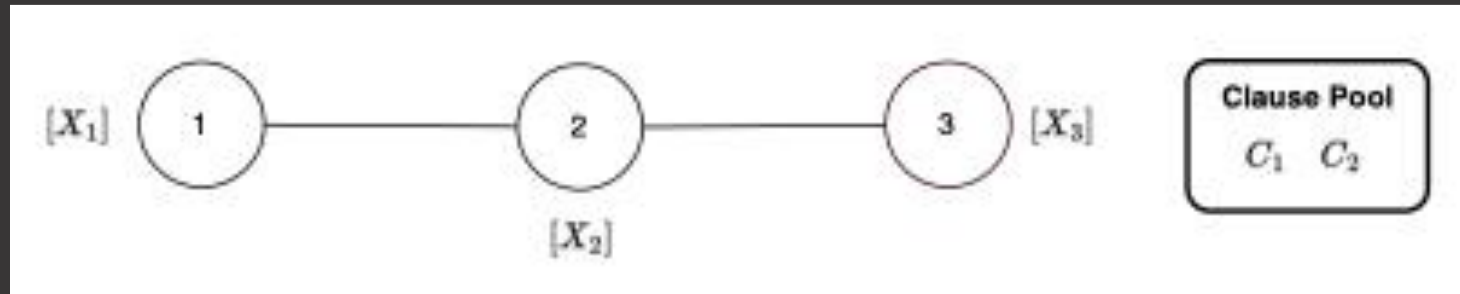
Clause-based node evaluation



$C_1(X_t) = \text{Eat}$

Clause ID Node ID

Nested (deep) clauses



$[\neg Eat, Hungry]$

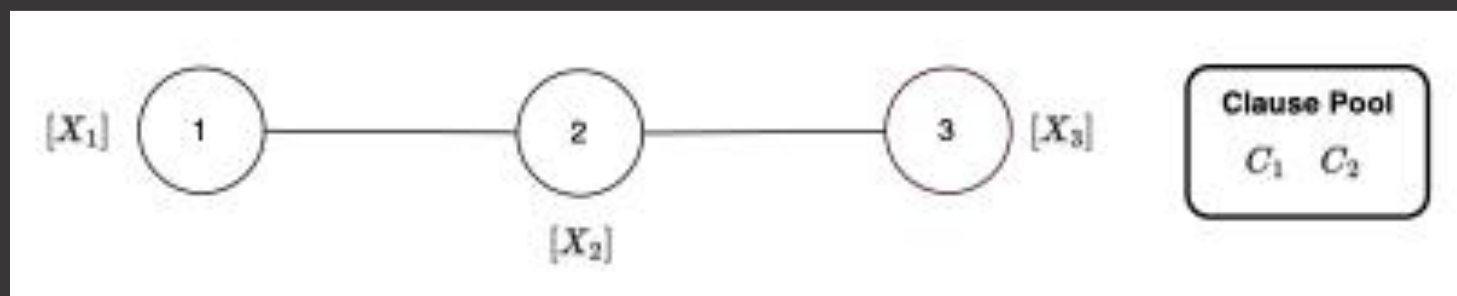
$[Eat, Hungry]$

$[\neg Eat, \neg Hungry]$

$$C_1(X_t) = Eat$$

$$C_2(X_t) = \neg Hungry \wedge C_1(X_{t-1})$$

Bidirectionality for multiple perspectives



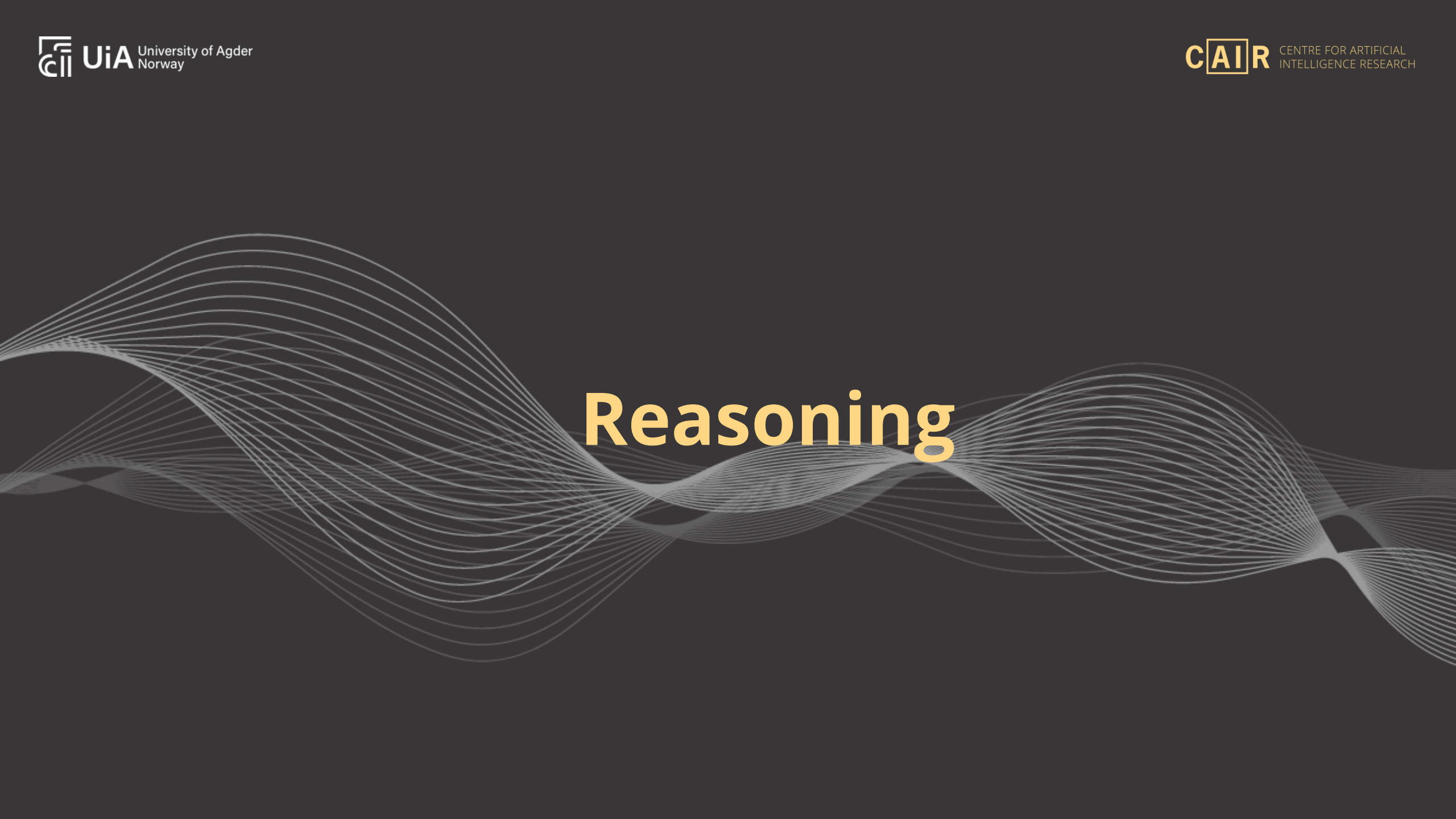
$[\neg Eat, Hungry]$

$[Eat, Hungry]$

$[\neg Eat, \neg Hungry]$

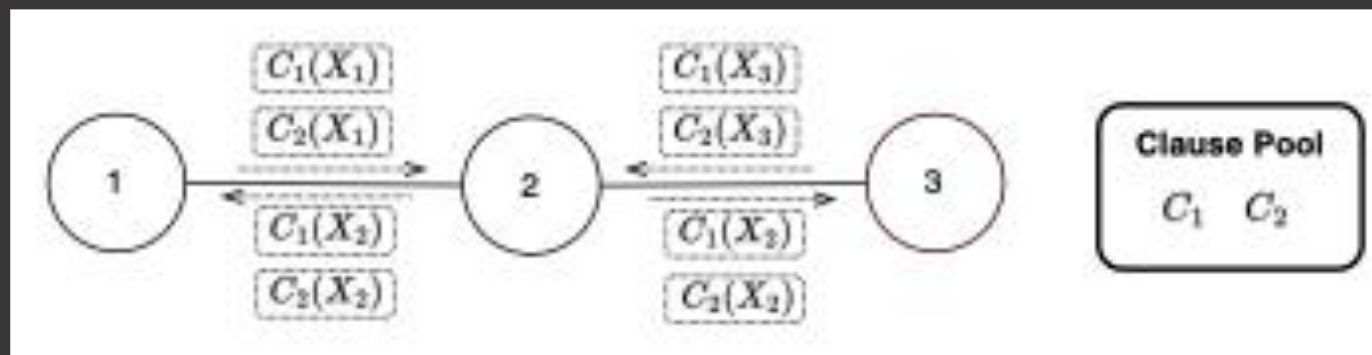
$$C_1(X_t) = Eat \wedge C_2(X_{t+1})$$

$$C_2(X_t) = \neg Hungry \wedge C_1(X_{t-1})$$



Reasoning

Clause-driven message passing



$[\neg Eat, Hungry]$

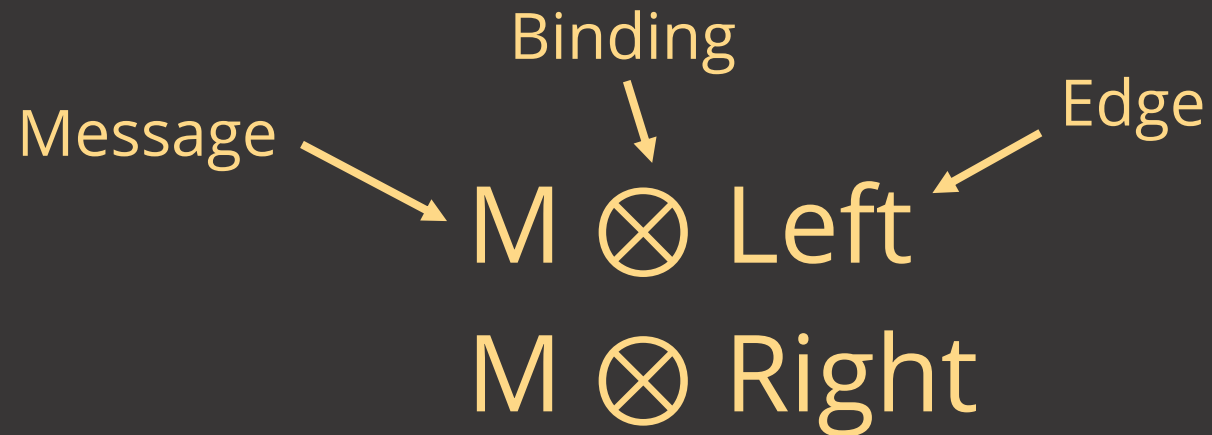
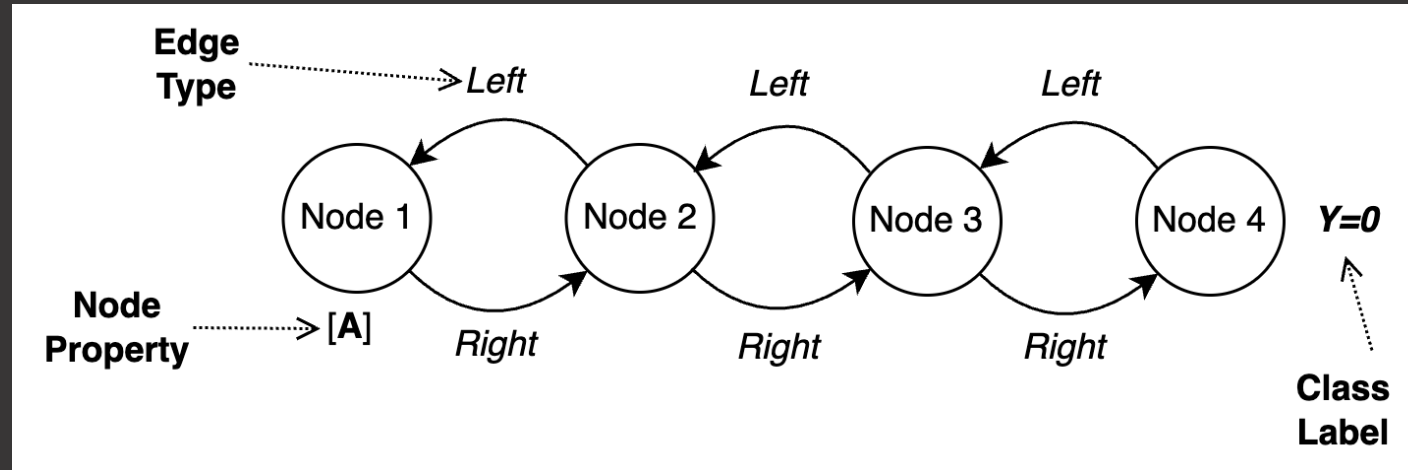
$[Eat, Hungry]$

$[\neg Eat, \neg Hungry]$

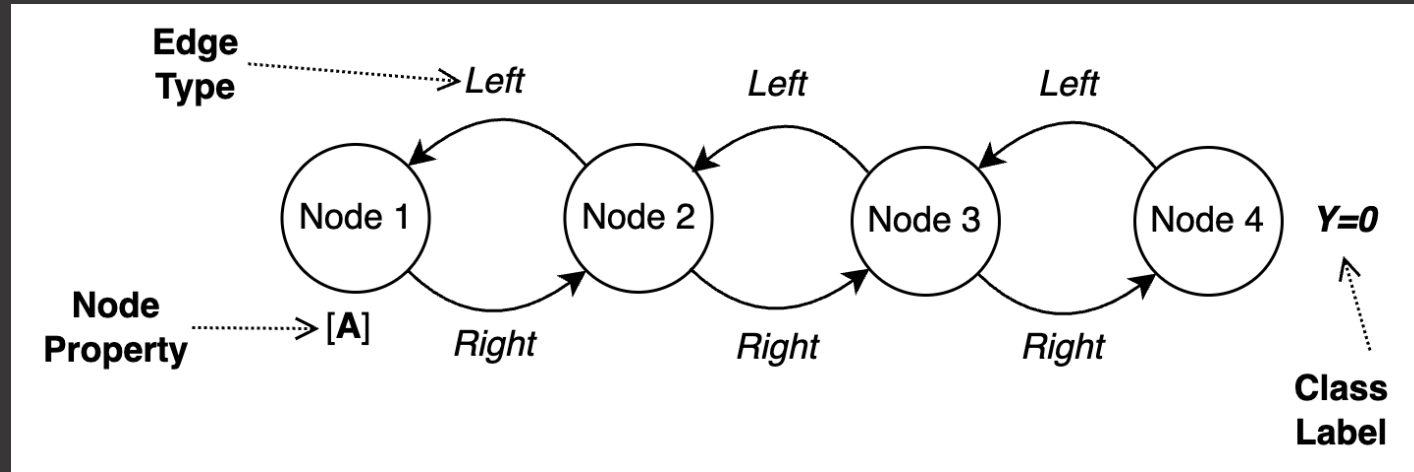
$$C_1(X_t) = \overset{\text{Attention}}{\text{Eat}} \wedge C_2(X_{t+1})$$

$$C_2(X_t) = \overset{\text{Attention}}{\neg Hungry} \wedge C_1(X_{t-1})$$

Vector symbolic edge types



Vector symbolic edge types



Edge type ID

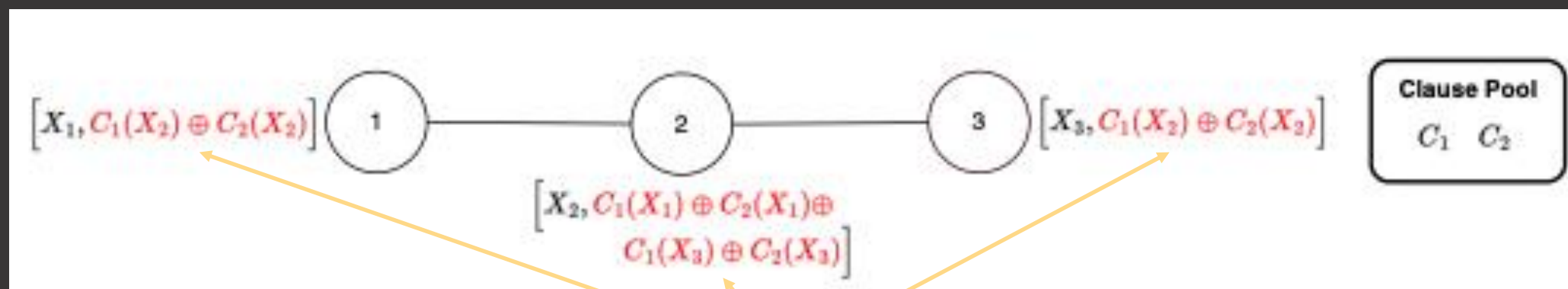
$Left := 1$
 $Right := 2$

Message

Binding

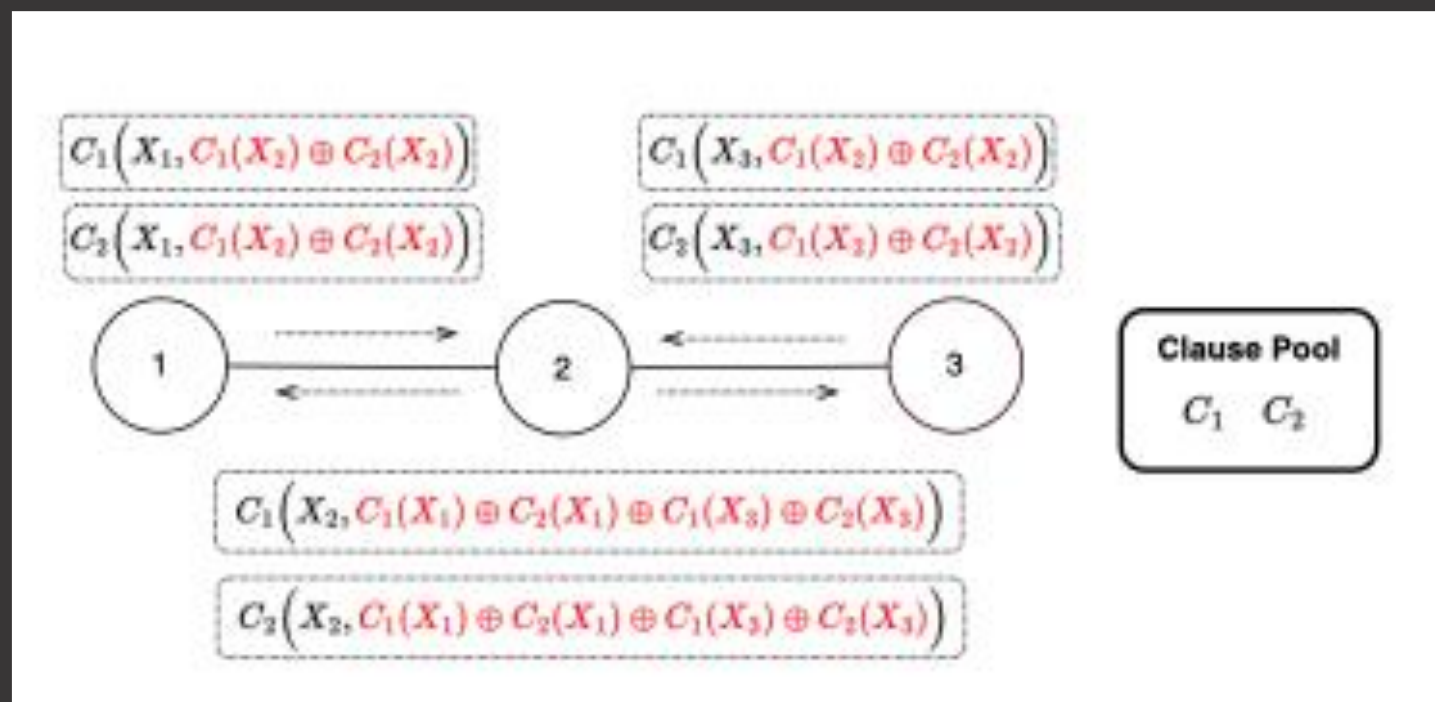
$M := [0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0]$
 $M \ll Left := [0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0]$
 $M \ll Right := [0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0]$

Contextualization



Messages stored as hypervectors

Round two of message-passing



Increased attention span

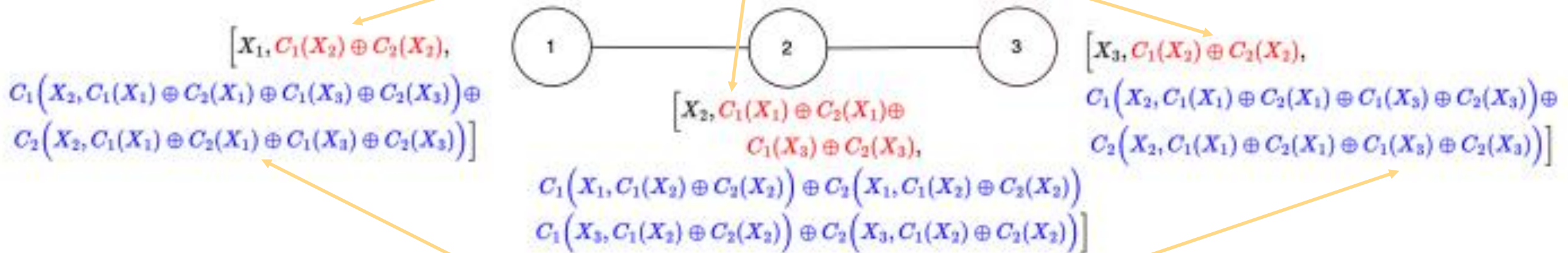
$$C_1(X_t) = \text{Eat} \wedge C_2(X_{t+1})$$

Increased attention span

$$C_2(X_t) = \neg \text{Hungry} \wedge C_1(X_{t-1})$$

Round two of contextualization

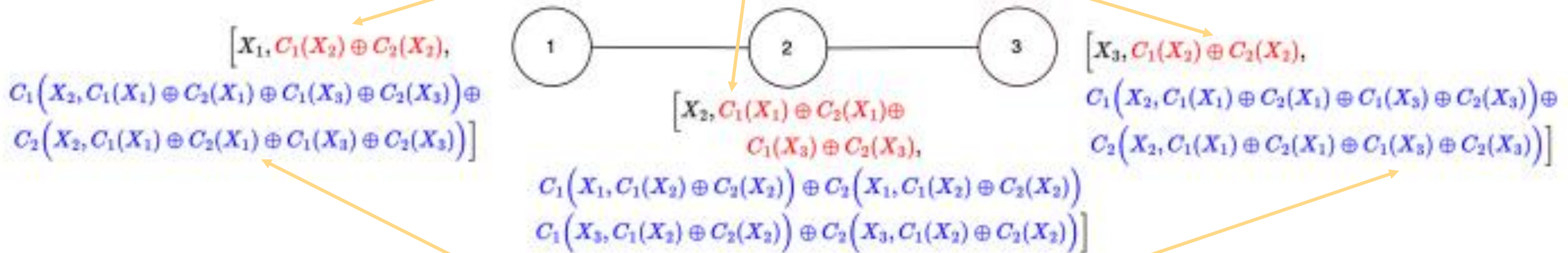
Messages from round one



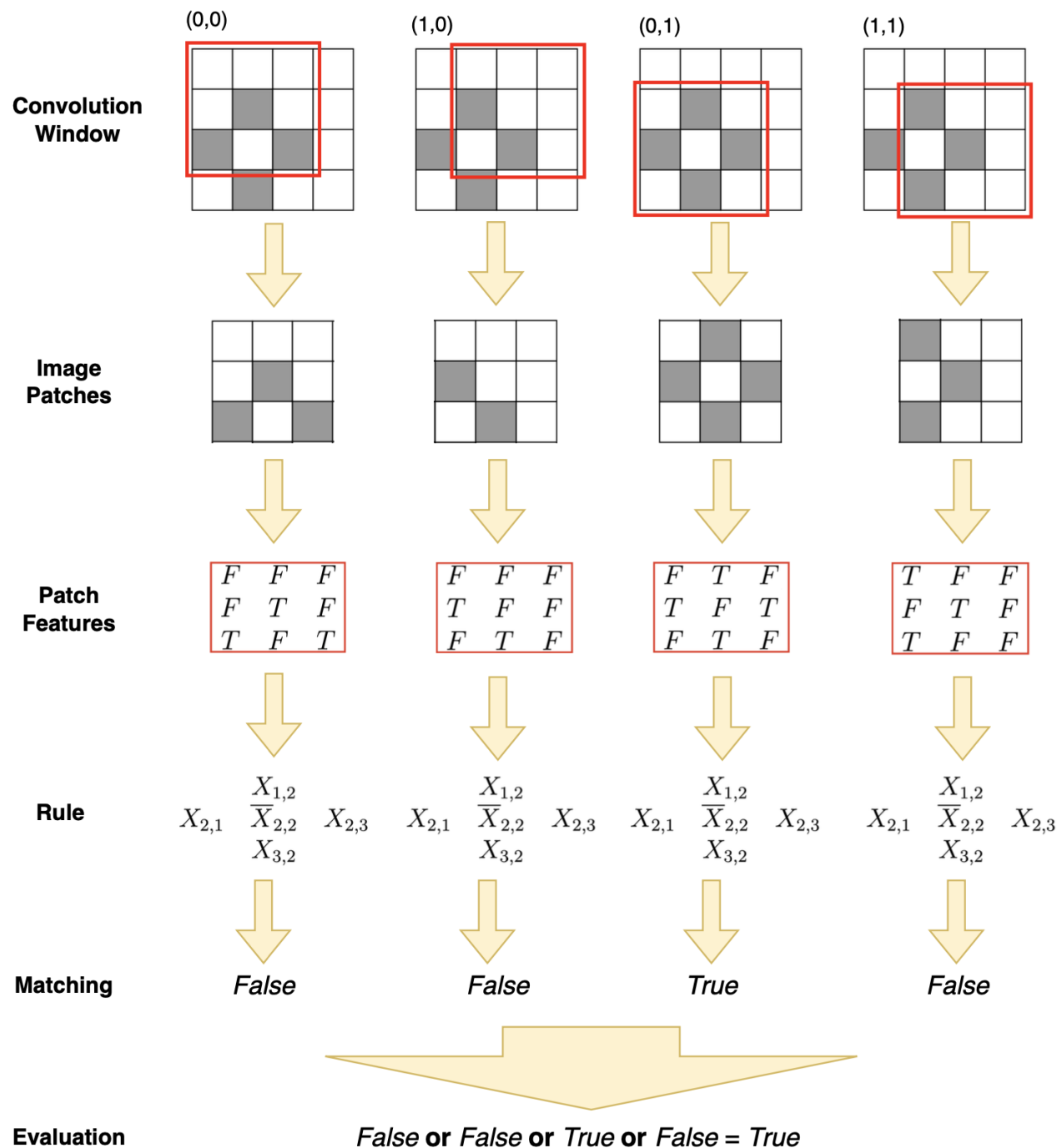
Messages from round two

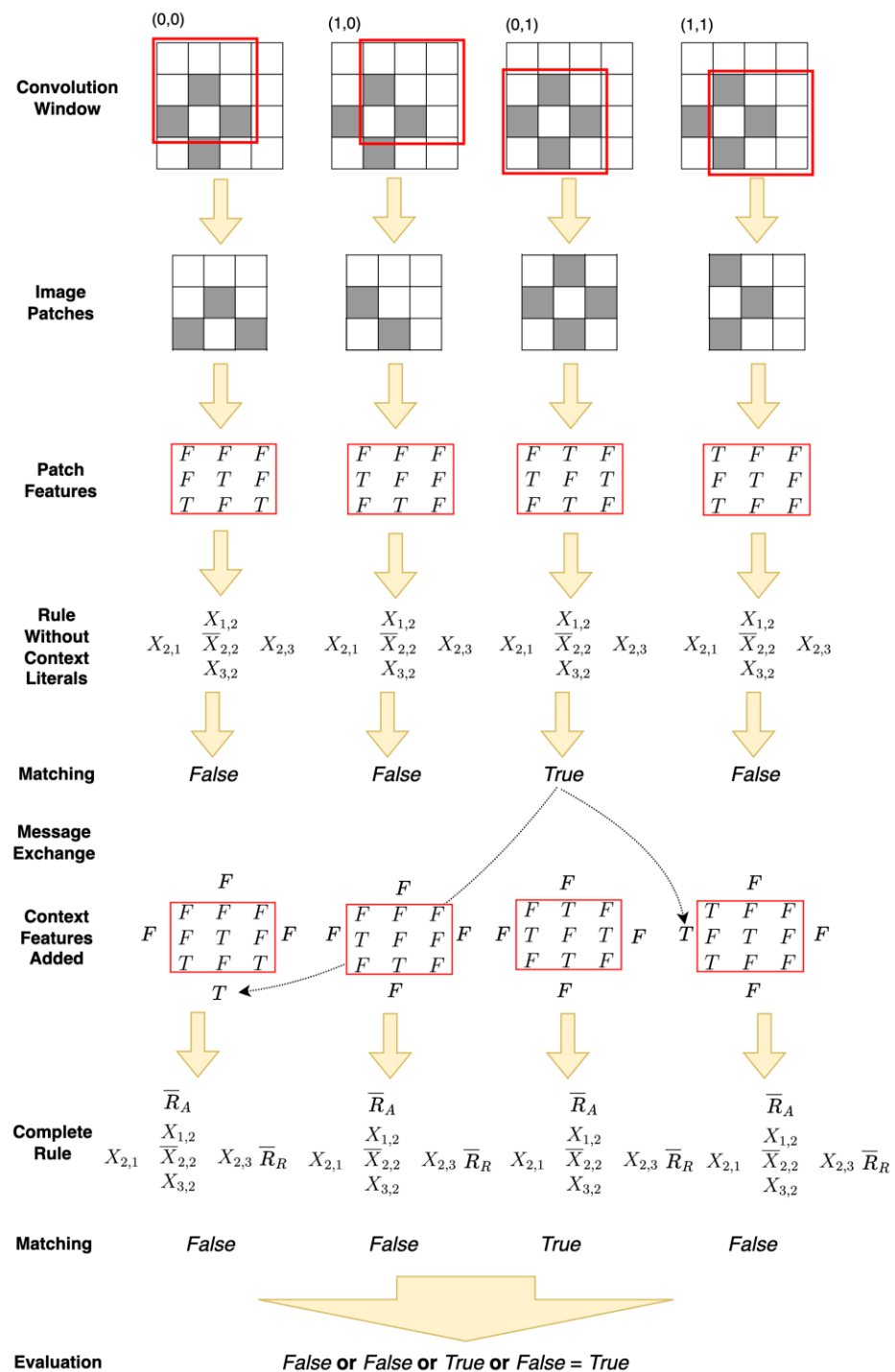
Round two of contextualization

Messages from round one

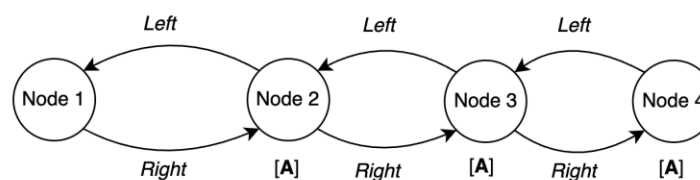


Messages from round two

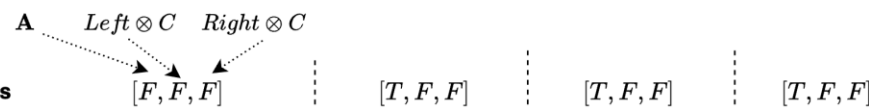




1) Input Graph



2) Initial Features



3) Clause Without Message Literals

$$\begin{aligned} & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \\ & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \\ & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \\ & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \end{aligned}$$

4) Partial Clause Matching

False True True True

5) Message Passing

6) Updated Features

[F, F, T] [T, F, T] [T, T, T] [T, T, F]

7) Full Clause

$$\begin{aligned} & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \\ & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \\ & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \\ & \mathbf{A} \wedge (\text{Left} \otimes C) \wedge (\text{Right} \otimes C) \end{aligned}$$

8) Full Clause Matching

False False True False

9) Evaluation

False or False or True or False = True

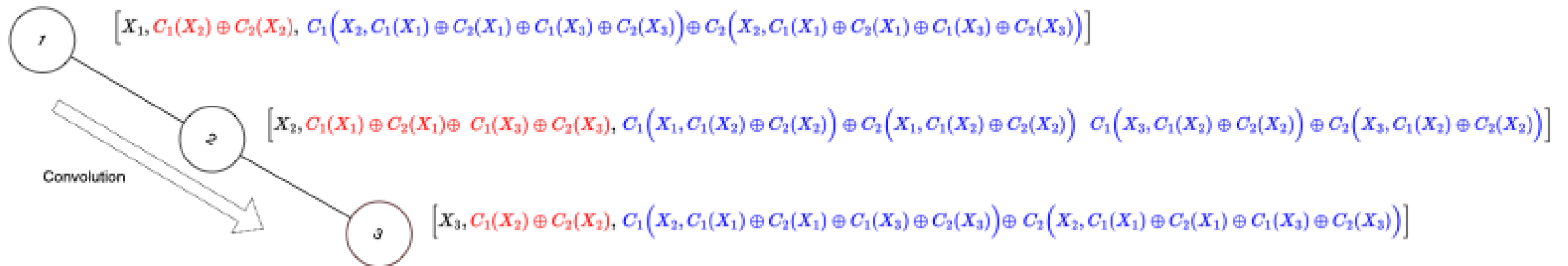
10) Classification

$Y = 2$ (Three As)



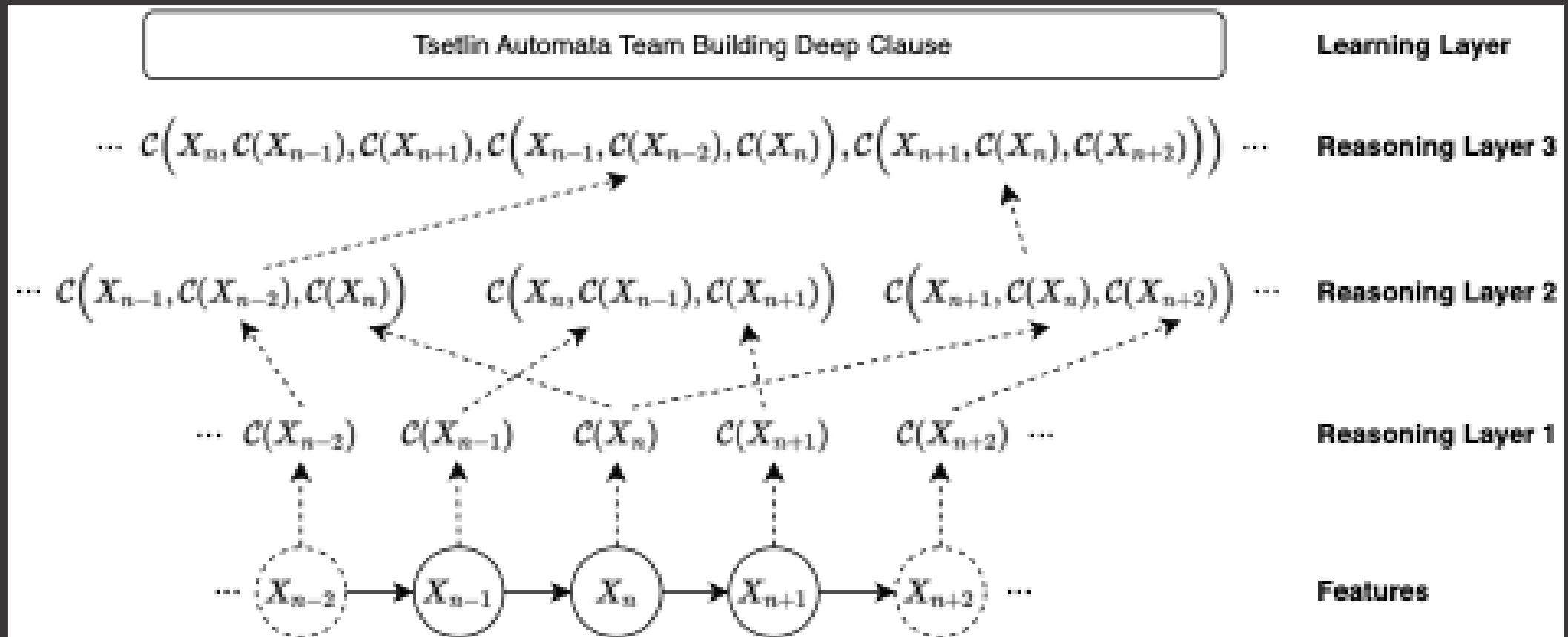
Deep logical learning

Tsetlin automata team building deep clause

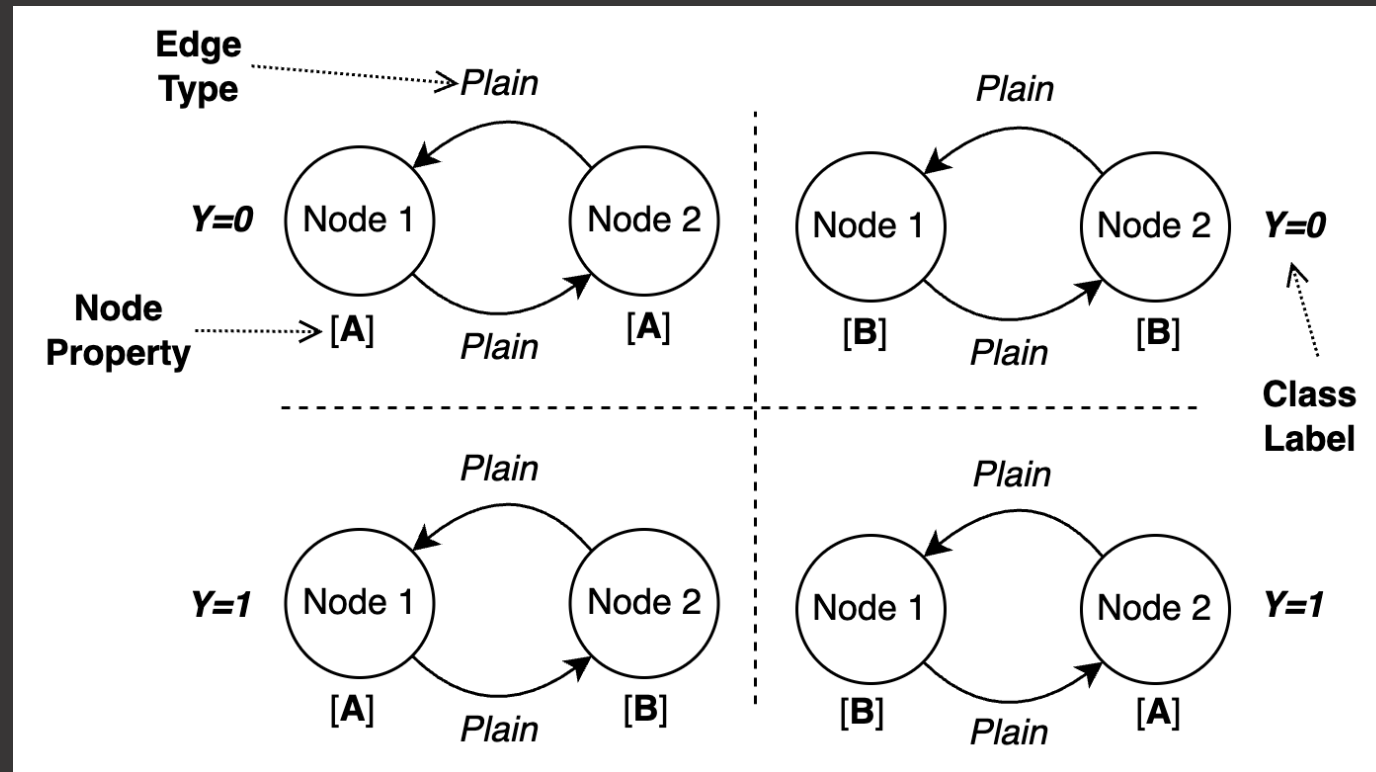


Tsetlin Automata Team Building Deep Clause

Single learning layer, multiple reasoning layers



Noisy XOR – interesting observation



Noisy XOR – interesting observation

$Y = 0$ $Y = 1$



Clause #0 W:(120 -131)

Clause #1 W:(-52 36) x5 AND NOT x14

Clause #2 W:(-31 45) x5 AND NOT x14

Clause #3 W:(-80 92) x14 AND NOT x5

A := [0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]

B := [0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0]

Use case - national early warning score

Graph Tsetlin Machine Example Use Case

